

Git & GitHub for DevOps — Complete Teaching Package

A beginner-friendly, 5-year-old-understandable, production-ready curriculum and reference for teaching Git & GitHub to DevOps students.

What Is This?

This is a **complete teaching package** designed for DevOps instructors. It includes:

- **3 Complete Curricula** — Choose the teaching style that fits your class
 - **1 DevOps Reference Bible** — A comprehensive manual for students to keep
 - **Step-by-step commands** with expected outputs
 - **Beginner-friendly analogies** that scale to professional terminology
 - **Real DevOps context** — Not just "how Git works" but "how DevOps teams actually use Git"
-

□ Directory Structure

```
/home/greg/learnkimchi/
├── README.md                    ← You are here (navigation & quick start)
├── git-github-devops-bible.md   ← The Reference Manual
├── approach-1-linear/
│   └── curriculum.md           ← Theory first, then practice
├── approach-2-project-driven/
│   └── curriculum.md           ← One continuous project, start to finish
└── approach-3-story-mode/
    └── curriculum.md           ← DevOps team onboarding story (recommended)
```

□ Quick Start — How to Use This Package

For Instructors

Step 1 — Pick your approach based on your students:

If your students are...	Use	Description
Complete beginners who need structure	Approach 1	Traditional lecture → lab → exercise format
Hands-on learners who want a real result	Approach 2	Students build one portfolio website across all sessions
Easily bored, need engagement/narrative	Approach 3 (recommended)	Students role-play joining a DevOps team; each session is a "day at work"

Step 2 — Assign environment setup BEFORE class:

- Students need Git, GitHub account, SSH keys, and a text editor
- This takes 20-30 minutes; don't use class time unless necessary
- See: [Environment Setup](#) below

Step 3 — Deliver the 5 sessions:

- Each session = exactly 60 minutes
- Each session has: Slides → Lab → Exercise → Review
- All commands include the **expected output** — students know if they're on track

Step 4 — Hand out the Bible:

- `git-github-devops-bible.md` is the reference students keep forever
- It's organized like a textbook: look up concepts, commands, or errors by topic

For Self-Learners / Students

Step 1 — Set up your environment (see below).

Step 2 — Pick one approach and follow it session by session.

Step 3 — Keep `git-github-devops-bible.md` open as your reference.

- When you forget a command, look at the cheat sheet (Part 12)
- When something breaks, check the troubleshooting guide (Part 11)

Step 4 — Build your portfolio.

- By the end, you'll have a GitHub repo with CI/CD — show it in job interviews

⚙️ Environment Setup (Do This First)

Analogy: Before you can drive, you need a license and keys. This section gets you licensed.

Step 1: Install Git

If you have Windows

1. Go to `https://git-scm.com/download/win`
2. Download and run the installer
3. Click **Next** repeatedly (defaults are fine)
4. Open **Git Bash** (search Start menu)

Check:

```
$ git --version
```

Expected: `git version 2.43.0...` (or similar)

If you have Mac

1. Open **Terminal** (Cmd+Space, type `terminal`)
2. Run:

```
$ git --version
```

Expected: `git version 2.39...` (or similar)

If it says "not installed," click **Install** when macOS prompts you.

If you have Linux

```
$ sudo apt update && sudo apt install git -y    # Ubuntu/Debian
# OR
$ sudo dnf install git -y                        # Fedora/RHEL

$ git --version
```

Expected: `git version 2.34...` (or similar)

Step 2: Tell Git Who You Are

```
$ git config --global user.name "Your Full Name"
$ git config --global user.email "your.email@example.com"
```

Important: Use the SAME email for GitHub in the next step.

Verify:

```
$ git config --global user.name  
$ git config --global user.email
```

Step 3: Create a GitHub Account

1. Go to `https://github.com`
2. Click **Sign up**
3. Use the **same email** as Step 2
4. Complete verification

Step 4: Set Up SSH Keys (The Secret Handshake)

This lets your computer talk to GitHub without a password.

```
$ ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press **Enter** three times (accept defaults, no password).

Copy your key:

- **Mac:** `pbcopy < ~/.ssh/id_ed25519.pub`
- **Windows (Git Bash):** `cat ~/.ssh/id_ed25519.pub | clip`
- **Linux:** `cat ~/.ssh/id_ed25519.pub` (select and copy)

Add to GitHub:

1. `https://github.com` → **Settings**
2. **SSH and GPG keys** → **New SSH key**
3. Title: `My laptop`
4. Paste the key
5. Click **Add SSH key**

Test it:

```
$ ssh -T git@github.com
```

Type `yes` when asked.

Expected: `Hi yourusername! You've successfully authenticated...`

Step 5: Install a Text Editor

- **Recommended:** VS Code (<https://code.visualstudio.com>)
 - **Alternatives:** Notepad++ (Windows), TextEdit (Mac), Nano (Linux)
-

□ The Three Approaches Explained

Approach 1: Linear Lecture + Lab

Style: Theory first, then practice

Best for: Students who need structure and clear boundaries

Each session:

1. **Lecture** (20 min) — Slides and analogies
2. **Guided Lab** (25 min) — Instructor demos, students follow
3. **Independent Exercise** (10 min) — Students work alone
4. **Review** (5 min) — Q&A

Topics per session:

- Session 1: Git basics (init, add, commit, status, log)
- Session 2: Branching and merging
- Session 3: GitHub, push, pull, Pull Requests
- Session 4: Undoing, reverting, conflict resolution
- Session 5: CI/CD with GitHub Actions

Approach 2: Project-Driven (Build-Along)

Style: One continuous project

Best for: Students who learn by building real things

Students build `my-devops-portfolio` across all 5 sessions:

- Session 1: Create repo, build Home page
- Session 2: Add About + Projects pages using branches
- Session 3: Push to GitHub, add Contact page via PR
- Session 4: Fix a bug, handle a merge conflict
- Session 5: Add CI/CD pipeline that validates the site

By the end, every student has a portfolio repo with CI/CD.

Approach 3: Story-Mode (Recommended)

Style: DevOps team onboarding narrative

Best for: Students who need engagement and context to remember

Students role-play joining "TechCorp" as a new DevOps engineer:

- Session 1: "Day 1 at the Company" — Set up repo, first commits
- Session 2: "Fixing Your First Bug" — Branching to fix a typo
- Session 3: "Submitting Your Work" — Push to GitHub, open first PR
- Session 4: "The Deploy Broke!" — Revert a bad commit, resolve conflicts
- Session 5: "Automating Everything" — Build a CI robot with GitHub Actions

Each session is a chapter in a story. Theory is introduced because the story needs it.

📖 The DevOps Bible

`git-github-devops-bible.md` is your **forever reference**. It is organized like a textbook:

Part	Topic	When to Read
Part 0	Environment Setup	Before you start
Part 1	Core Concepts	If you're new to Git
Part 2	Essential Daily Workflow	Every day — the commands you actually use
Part 3	Branching & Merging	When working with branches
Part 4	GitHub Workflow	When pushing code or opening PRs
Part 5	History & Undoing	When you make a mistake
Part 6	Collaboration	When working with a team
Part 7	DevOps Branching Patterns	When designing team workflows
Part 8	CI/CD with GitHub Actions	When automating deploys
Part 9	Release Management	When shipping versions
Part 10	Advanced Topics	When you're comfortable and want more
Part 11	Troubleshooting Guide	WHEN SOMETHING BREAKS
Part 12	Command Cheat Sheet	When you forgot the exact flag

□ Common Workflows

The Solo Dev Workflow

```
# Start of day
$ cd my-project
$ git switch main
$ git pull

# Create feature branch
$ git switch -c feature/my-change

# Do work... edit files...

# Commit
$ git add .
$ git commit -m "Clear description of what changed"

# Push
$ git push -u origin feature/my-change

# Open Pull Request on GitHub, merge, delete branch

# Back to main
$ git switch main
$ git pull
$ git branch -d feature/my-change
```

The "Oh No, Production Is Down" Workflow

```
# Find the bad commit
$ git log --oneline

# Revert it
$ git revert BAD_COMMIT_ID --no-edit

# Push the fix
$ git push

# Check that it's fixed
```

The "I Messed Up But Haven't Pushed Yet" Workflow

```
# Wrong files staged
$ git restore --staged wrong-file.txt

# Bad commit message (last commit only)
$ git commit --amend -m "Better message"

# Committed to wrong branch
$ git switch -c correct-branch # Brings the commit with you
$ git switch wrong-branch
$ git reset --soft HEAD~1
```

The "My Team Has Changed Main" Workflow

```
# Download their changes
$ git switch main
$ git pull

# Update your feature branch
$ git switch feature/my-change
$ git rebase main

# Fix any conflicts, then push
$ git push --force-with-lease
```

FAQ

Q: Do I need to know programming?

A: No. This course assumes zero coding knowledge.

Q: Can I skip the environment setup?

A: No. Everything else depends on it.

Q: Which approach is best?

A: Approach 3 (Story-Mode) is the most engaging for beginners. Approach 1 is best for very structured learners. Approach 2 is best for those who want a portfolio piece at the end.

Q: How long does each session take?

A: Exactly 60 minutes if you follow the pacing.

Q: Can I teach this in one day instead of five?

A: You can try, but beginners need time to absorb. If you must compress, skip the extended exercises and do them as demos.

Q: Do students need a paid GitHub account?

A: No. Everything works on free GitHub accounts.

Q: What if a student uses Windows and another uses Mac?

A: The commands are the same. Only the terminal application differs (Git Bash vs. Terminal).

Q: What if a student gets stuck?

A: Check Part 11 of the Bible (Troubleshooting Guide). Every common error is listed with a fix.

Q: Can I add my own company examples?

A: Absolutely. Replace "TechCorp" in Approach 3 with your company name.

Q: Where do the analogies come from?

A: Each analogy is designed to be understood by a 5-year-old, then mapped to the real DevOps concept.

□ When Something Breaks

Don't panic. Go to `git-github-devops-bible.md` → **Part 11: Troubleshooting Guide**.

Common issues covered:

- "fatal: not a git repository"
 - "Permission denied"
 - Merge conflicts
 - "Your branch has diverged"
 - Accidentally committed secrets
 - Failed GitHub Actions workflows
-

☐ Capstone Checklist

By the end of all 5 sessions, every student should be able to demonstrate:

- [] Created a Git repository from scratch
 - [] Made commits with meaningful messages
 - [] Created and merged branches
 - [] Connected a local repo to GitHub
 - [] Pushed and pulled code
 - [] Opened and merged a Pull Request
 - [] Reverted a bad commit
 - [] Resolved a merge conflict
 - [] Written a GitHub Actions workflow
 - [] Set up branch protection
 - [] Explained what CI/CD means using an analogy
-

☐ License & Usage

You are free to use, modify, and distribute these materials for teaching purposes. Attribution appreciated but not required.

☐ Need Help?

If you're stuck:

1. Check the **Bible** → Part 11 (Troubleshooting)
 2. Check the **Bible** → Part 12 (Cheat Sheet)
 3. Run `git status` — it usually tells you what to do next
-

Happy teaching. May all your commits be clean and all your pipelines green.

The Git & GitHub DevOps Reference Manual

A Complete Bible for DevOps Engineers

Version: 1.0
Audience: DevOps engineers from absolute beginner to advanced
Philosophy: Every concept explained simply enough for a five-year-old, then scaled to production-grade engineering.

Table of Contents

Module	Topic	Page
Part 0	Before You Start: Environment Setup	1
Part 1	Core Concepts	2
Part 2	The Essential Daily Workflow	4
Part 3	Branching & Merging	7
Part 4	GitHub Workflow: Push, Pull, PRs	10
Part 5	History, Undoing & Recovery	13
Part 6	Collaboration: Conflicts, Stash & Rebase	16
Part 7	DevOps Branching Patterns	19
Part 8	CI/CD with GitHub Actions	21
Part 9	Release Management: Tags & Releases	24
Part 10	Advanced Topics	26
Part 11	Troubleshooting Guide	28
Part 12	Command Cheat Sheet	30

How to Use This Manual

1. **New to Git?** Start at [Part 0](#) and go in order.
2. **Know the basics?** Jump to [Part 5](#) for undoing, or [Part 6](#) for team collaboration.
3. **Setting up CI/CD?** Go straight to [Part 8](#).
4. **Something broke?** Check [Part 11](#).
5. **Need a quick reminder?** Flip to [Part 12](#).

Every command in this manual shows:

- **Environment:** Where to run it (Terminal / Browser / GitHub UI)
 - **Expected Output:** Exactly what you should see
 - **Common Errors:** What goes wrong and how to fix it
 - **DevOps Context:** Why this matters in production
-
-

Part 0: Before You Start — Environment Setup

Analogy: Before you can drive a car, you need a driver's license, keys, and to know which pedal is which. This section gets you licensed and seated.

0.1 Install Git

Environment: Windows

1. Visit `https://git-scm.com/download/win`
2. Download the installer
3. Run it. Accept all defaults (click **Next** repeatedly)
4. Open **Git Bash** (Start menu → search "Git Bash")

Verify:

```
$ git --version
```

Expected Output:

```
git version 2.43.0.windows.1
```

(Your version number may differ)

Environment: macOS

Open **Terminal** (Cmd+Space, type `terminal`):

```
$ git --version
```

Expected Output:

```
git version 2.39.3
```

If not found, macOS will prompt to install **Command Line Developer Tools**. Click **Install**.

Environment: Linux (Ubuntu / Debian)

```
$ sudo apt update && sudo apt install git -y  
$ git --version
```

Expected Output:

```
git version 2.34.1
```

Environment: Linux (Fedora / CentOS / RHEL)

```
$ sudo dnf install git -y  
$ git --version
```


0.2 Configure Your Identity

Git needs to know who you are so it can label your work.

Environment: Any terminal

```
$ git config --global user.name "Your Full Name"
$ git config --global user.email "your.email@example.com"
```

***Why?** Every commit is signed with your name and email. In a company, this creates an audit trail.*

Verify:

```
$ git config --global user.name
$ git config --global user.email
```

IMPORTANT: Use the **same email** for GitHub.

0.3 Create a GitHub Account

Environment: Web Browser

1. Go to `https://github.com`
2. Click **Sign up**
3. Enter your email, create a password and username
4. Complete email verification

0.4 Set Up SSH Authentication

SSH lets your computer talk to GitHub securely without typing your password every time.

Environment: Terminal

```
$ ssh-keygen -t ed25519 -C "your.email@example.com"
```

Press **Enter** three times (accept default location, no passphrase).

Copy your public key:

macOS:

```
$ pbcopy < ~/.ssh/id_ed25519.pub
```

Windows (Git Bash):

```
$ cat ~/.ssh/id_ed25519.pub | clip
```

Linux:

```
$ cat ~/.ssh/id_ed25519.pub
```

(Select and copy the output)

Add to GitHub:

1. <https://github.com> → Click your profile picture → **Settings**
2. Left sidebar: **SSH and GPG keys**
3. Click **New SSH key**
4. Title: `My Laptop`
5. Key: Paste what you copied
6. Click **Add SSH key**

Test the connection:

```
$ ssh -T git@github.com
```

Type `yes` when asked about authenticity.

Expected Output:

```
Hi yourusername! You've successfully authenticated...
```

0.5 Install a Text Editor

- **Recommended:** VS Code (<https://code.visualstudio.com>)
- **Alternatives:** Notepad++ (Windows), TextEdit (Mac), Nano/Vim (Linux)

0.6 Configure Default Branch Name

Modern Git and GitHub use `main` instead of `master`. Ensure consistency:

```
$ git config --global init.defaultBranch main
```

0.7 Configure Line Endings (Windows Only)

Windows uses different line endings than Linux/Mac. Fix this:

Environment: Windows (Git Bash)

```
$ git config --global core.autocrlf true
```

Environment: macOS / Linux

```
$ git config --global core.autocrlf input
```

Part 1: Core Concepts

1.1 What is Version Control?

5-Year-Old Analogy: *Imagine a magic camera. You build something with blocks, then take a photo. If your baby brother knocks it over, you show the photo to the magic camera and say "Put it back like this!" The camera restores your blocks exactly.*

DevOps Translation: *Version control tracks every change to files, who made it, when, and why. If production breaks, you can restore the exact code from before the break.*

1.2 Key Concepts Dictionary

Simple Term	Technical Term	What It Actually Is
The folder with the camera inside	Repository (Repo)	A directory containing a <code>.git</code> folder that tracks everything
A photo of your work	Commit	A permanent snapshot of your files at one moment in time
The box where you put things before the photo	Staging Area (Index)	A temporary holding area for files you want to include in the next commit
Your messy desk	Working Directory	Your actual files as they exist right now
The photo album	History	All commits, connected like a timeline
A copy of the photo album	Remote	A version of the repo stored on another server (GitHub)
A parallel universe	Branch	An independent line of development
Combining two universes	Merge	Joining one branch's changes into another
Permission slip	Pull Request (PR)	Requesting to merge your branch into main, with code review

1.3 The Three States of Git

Every file in Git is in one of three states:

Working Directory (modified)	--git add--> stage	Staging Area (staged)	--git commit--> commit	Git Repository (History) (committed)
---------------------------------	-----------------------	--------------------------	---------------------------	---

1. **Working Directory:** You edit files here. Git notices changes but doesn't save them yet.
2. **Staging Area:** You choose which changes to include in your next snapshot.
3. **Repository:** Committed snapshots live here forever.

Restaurant Analogy:

- Working Directory = The prep table where you're chopping vegetables

- *Staging Area = The plate the chef arranged and is about to serve*
- *Commit = The photo that goes on the menu (permanent record)*

1.4 The Lifecycle of a File

Untracked (new)	--git add-->	Staged (ready)	--git commit-->	Unmodified (saved)	--edit-->	Modified (updated)
--------------------	--------------	-------------------	-----------------	-----------------------	-----------	-----------------------

1.5 Understanding `.git`

When you run `git init`, Git creates a hidden folder called `.git`.

This is Git's brain. It stores:

- *Every commit ever made*
- *Every branch*
- *Configuration*
- *The staging area*

NEVER delete `.git` unless you want to erase all history.

To see it:

```
$ ls -la
```

Expected Output:

```
drwxr-xr-x  .git
-rw-r--r--  yourfile.txt
```

Part 2: The Essential Daily Workflow

These are the commands you will use every single day as a DevOps engineer.

2.1 Creating a New Repository

Environment: Terminal

```
$ mkdir my-project  
$ cd my-project  
$ git init
```

Expected Output:

```
Initialized empty Git repository in /home/you/my-project/.git/
```

DevOps Context: You run `git init` when starting a new project, creating a new deployment script repo, or setting up infrastructure-as-code.

2.2 Checking Your Status

This is the single most important command. Run it constantly.

```
$ git status
```

Expected Output (clean repo):

```
On branch main  
nothing to commit, working tree clean
```

Expected Output (untracked file):

```
On branch main  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    new-file.txt
```

Expected Output (modified file):

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
   modified:   existing-file.txt
```

Expected Output (staged file):

```
On branch main
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
   new file:   new-file.txt
```

2.3 Adding Files to the Staging Area

Add one file:

```
$ git add filename.txt
```

Add all files in current directory:

```
$ git add .
```

Add all modified files:

```
$ git add -A
```

DevOps Context: Stage files intentionally. Don't blindly `git add .` — review what you're committing first.

2.4 Unstaging Files

Oops, you staged the wrong file:

```
$ git restore --staged filename.txt
```

Before (staged):

```
Changes to be committed:
  new file:   filename.txt
```

After:

```
Untracked files:
  filename.txt
```

2.5 Committing Your Work

```
$ git commit -m "Describe what you changed"
```

Example — Good messages:

```
$ git commit -m "Add nginx configuration for production"
$ git commit -m "Fix database connection timeout in staging"
$ git commit -m "Update Terraform module to v3.2.1"
```

Example — Bad messages:

```
$ git commit -m "asdf"           # Meaningless
$ git commit -m "stuff"          # What stuff?
$ git commit -m "fixed bug"      # Which bug?
$ git commit -m "update"         # Update what?
```

Multi-line commit (for detailed changes):

```
$ git commit -m "Subject line under 50 chars" -m "Body: detailed explanation of what"
```

DevOps Context: In incident response, clear commit messages save hours. "Fix load balancer health check" is infinitely more useful than "fix stuff."

2.6 Viewing History

Full log:


```
$ git log
```

Compact one-line format (most useful):

```
$ git log --oneline
```

With graph (shows branch structure):

```
$ git log --oneline --graph --all
```

Last 5 commits:

```
$ git log --oneline -5
```

Show what changed in each commit:

```
$ git log --oneline --stat
```

Show file-by-file patch:

```
$ git log --oneline --patch
```

Search commits by message:

```
$ git log --oneline --grep="nginx"
```

View commits by a specific author:

```
$ git log --oneline --author="Your Name"
```

2.7 Seeing What Changed (Before Committing)

Show unstaged changes:

```
$ git diff
```

Show staged changes (what will be committed):

```
$ git diff --staged
```

Show changes in a specific file:

```
$ git diff filename.txt
```

2.8 Ignoring Files (`.gitignore`)

Create a `.gitignore` file to tell Git "never track these files."

Environment: Terminal

```
$ cat > .gitignore << 'EOF'
# Operating system files
.DS_Store
Thumbs.db

# Logs
*.log
logs/

# Environment variables (NEVER commit secrets)
.env
.env.local

# Dependencies
node_modules/
vendor/

# Build outputs
dist/
build/
*.exe

# IDE files
.vscode/
.idea/
*.swp

# Terraform
.terraform/
*.tfstate
*.tfstate.*
EOF

$ git add .gitignore
$ git commit -m "Add .gitignore for common files"
```

CRITICAL DevOps Rule: Never commit passwords, API keys, `.env` files, or Terraform state files. Use `.gitignore` to protect yourself.

2.9 Removing Files

Remove from Git AND delete from disk:

```
$ git rm old-file.txt
$ git commit -m "Remove deprecated configuration"
```

Remove from Git but KEEP on disk:

```
$ git rm --cached secret-file.txt
$ git commit -m "Remove accidentally committed secret"
```

Then add it to `.gitignore` immediately.

Part 3: Branching & Merging

3.1 What is a Branch?

5-Year-Old Analogy: You're reading a choose-your-own-adventure book. The main story is one path. You put a bookmark at page 50, try a different adventure, and if it's good, you rewrite the main story to include it. The bookmarked path = a branch.

DevOps Translation: Branches let you experiment safely. You NEVER edit production (`main`) directly. You create a branch, test your changes, then merge.

3.2 Basic Branch Commands

List branches:

```
$ git branch
```

Expected Output:

```
feature-auth
hotfix-timeout
* main
```

* = You are currently on this branch.

Create a new branch:

```
$ git branch feature-new-thing
```

Switch to a branch:

```
$ git switch feature-new-thing
```

Create AND switch (shortcut):

```
$ git switch -c feature-new-thing
```

(The `-c` flag means "create")

Delete a merged branch:

```
$ git branch -d feature-new-thing
```

Delete an unmerged branch (force):

```
$ git branch -D feature-new-thing
```

Rename current branch:

```
$ git branch -m better-name
```

3.3 Understanding HEAD

HEAD is a pointer to your current commit.

- HEAD = "You are here"
- HEAD~1 = "One commit before where you are"
- HEAD~3 = "Three commits before where you are"

3.4 Merging Branches

Fast-forward merge (main hasn't changed since you branched):

```
$ git switch main
$ git merge feature-new-thing
```

Expected Output:

```
Updating abc123..def456
Fast-forward
 newfile.txt | 10 ++++++++
 1 file changed, 10 insertions(+)
```

DevOps Context: *Fast-forward is clean and simple. No merge commit needed.*

Three-way merge (main has new commits):

```
$ git switch main
$ git merge feature-new-thing
```

Expected Output:

```
Merge made by the 'ort' strategy.
 newfile.txt | 10 ++++++++
 1 file changed, 10 insertions(+)
```

This creates a **merge commit** — a new commit that has two parents.

Avoid merge commits with rebase (see Part 6).

3.5 Branch Naming Conventions

Pattern	Example	When to Use
feature/description	feature/user-auth	New features
bugfix/description	bugfix/login-timeout	Bug fixes
hotfix/description	hotfix/critical-db-fix	Production emergencies
release/version	release/v2.1.0	Release preparation
docs/description	docs/api-examples	Documentation updates

3.6 Comparing Branches

See what `feature` has that `main` doesn't:

```
$ git log main..feature --oneline
```

See differences between branches:

```
$ git diff main..feature
```

3.7 The Complete Branch Workflow

```
# 1. Start from latest main
$ git switch main
$ git pull

# 2. Create feature branch
$ git switch -c feature/my-change

# 3. Do your work
# ... edit files ...

# 4. Commit
$ git add .
$ git commit -m "Add my feature"

# 5. Push branch
$ git push -u origin feature/my-change

# 6. (On GitHub) Open Pull Request → Review → Merge

# 7. Pull latest main
$ git switch main
$ git pull

# 8. Delete local branch
$ git branch -d feature/my-change
```

Part 4: GitHub Workflow — Push, Pull, PRs

4.1 Connecting to a Remote

Add a remote (typically done once per repo):

```
$ git remote add origin git@github.com:USERNAME/REPO.git
```

Verify remotes:

```
$ git remote -v
```


Expected Output:

```
origin git@github.com:USERNAME/REPO.git (fetch)
origin git@github.com:USERNAME/REPO.git (push)
```

Change remote URL:

```
$ git remote set-url origin git@github.com:NEW-USER/NEW-REPO.git
```

4.2 Pushing Code

Push current branch to remote:

```
$ git push
```

Push a new branch for the first time:

```
$ git push -u origin feature-name
```

The `-u` flag (short for `--set-upstream`) links your local branch to the remote branch. After this, just `git push` works.

Force push (DANGEROUS — rewrites history):

```
$ git push --force
```

DevOps Warning: Only force push on personal feature branches. Never force push to `main` or shared branches.

Force push with lease (safer):

```
$ git push --force-with-lease
```

4.3 Pulling Code

Pull (fetch + merge):

```
$ git pull
```

Fetch only (download without merging):

```
$ git fetch
```

Fetch a specific branch:

```
$ git fetch origin feature-name
```

Pull with rebase (cleaner history):

```
$ git pull --rebase
```

4.4 Cloning a Repository

Clone a remote repo to your local machine:

```
$ git clone git@github.com:USERNAME/REPO.git
```

Clone into a specific folder:

```
$ git clone git@github.com:USERNAME/REPO.git custom-folder-name
```

Clone only the latest commit (faster for CI):

```
$ git clone --depth 1 git@github.com:USERNAME/REPO.git
```

4.5 The Pull Request (PR) Workflow

The PR workflow is the backbone of professional DevOps. No one pushes directly to `main`.

Step-by-Step:

1. Push your branch:

```
bash $ git push -u origin feature-name
```

2. Open PR on GitHub:

- Go to repo on <https://github.com>
- Click "**Compare & pull request**"
- Add title and description
- Click **Create pull request**

3. Code review happens:

- Team members review the **Files changed** tab
- They leave comments on specific lines
- You make fixes, commit, push (PR updates automatically)

4. CI runs (GitHub Actions):

- Tests must pass
- Linting must pass

5. Approve and merge:

- Reviewer clicks **Approve**
- Click **Merge pull request**
- Choose merge strategy:
 - **Create a merge commit** (preserves branch history)
 - **Squash and merge** (combines all commits into one)
 - **Rebase and merge** (replays commits on main)

6. Clean up:

- Delete the remote branch
- Pull locally:

```
bash $ git switch main $ git pull $ git branch -d feature-name
```

4.6 Forking vs. Cloning

	Clone	Fork
What	Copy to your laptop	Copy on GitHub to your account
When	Working on your team's repo	Contributing to open source
Example	<pre>git clone git@github.com:mycompany/ project.git</pre>	Click "Fork" on <code>github.com/ kubernetes/kubernetes</code>

After forking, clone your fork:

```
$ git clone git@github.com:YOUR-NAME/original-repo.git
```

4.7 Syncing a Fork

```
# Add the original repo as "upstream"  
$ git remote add upstream git@github.com:original-owner/original-repo.git  
  
# Fetch latest from original  
$ git fetch upstream  
  
# Merge into your main  
$ git switch main  
$ git merge upstream/main  
  
# Push to your fork  
$ git push
```

Part 5: History, Undoing & Recovery

5.1 Viewing History in Detail

Show commit with changes:

```
$ git show COMMIT_ID
```

Show only files changed:

```
$ git show COMMIT_ID --stat
```

Show who changed each line of a file:

```
$ git blame filename.txt
```

Expected Output:

```
abc1234 (Your Name  2024-01-15 09:00:00 +0000  1) Line one content  
def5678 (Teammate   2024-01-16 14:30:00 +0000  2) Line two content
```

DevOps Context: *git blame* is essential during incidents. "Who changed this config line 3 days ago?"

Search all commits for a string:

```
$ git log --all --grep="timeout"  
$ git log --all -S "database_url" # Search code content, not just messages
```

5.2 Reverting a Commit (Safe Undo)

Rule: *Revert is always safe. It creates a NEW commit that undoes an OLD commit. History is preserved.*

Revert the most recent commit:

```
$ git revert HEAD
```

Revert a specific commit:

```
$ git revert abc1234
```

An editor opens for the commit message. Save and exit.

Revert without editing commit message:

```
$ git revert HEAD --no-edit
```

Revert a merge commit:

```
$ git revert -m 1 MERGE_COMMIT_ID
```

DevOps Context: *Revert is your go-to for production incidents. It undoes a bad deploy instantly while keeping the audit trail.*

5.3 Resetting (Use With Caution)

Reset moves the branch pointer. It can erase commits.

Soft reset (keep changes staged):

```
$ git reset --soft HEAD~1
```

Mixed reset (keep changes unstaged):

```
$ git reset --mixed HEAD~1
```

Hard reset (DESTROY changes — DANGEROUS):

```
$ git reset --hard HEAD~1
```

DevOps Warning: `git reset --hard` permanently deletes uncommitted work. Never use on shared history.

5.4 The Reflog — Your Safety Net

Analogy: The reflog is like a backup camera that recorded everything, even things you thought you deleted.

View all actions (even after reset):

```
$ git reflog
```

Expected Output:

```
abc1234 HEAD@{0}: commit: Add feature
def5678 HEAD@{1}: commit: Fix bug
1234567 HEAD@{2}: reset: moving to HEAD~1
```

Recover a lost commit:

```
$ git reset --hard HEAD@{2}
```

View reflog for a specific branch:

```
$ git reflog show main
```

DevOps Context: `git reflog` has saved many engineers. If you accidentally reset `main`, the reflog remembers.

5.5 Cherry-Picking

Apply a specific commit from one branch to another.

```
# On main, apply a commit from feature
$ git switch main
$ git cherry-pick abc1234
```

Cherry-pick without committing (just apply):

```
$ git cherry-pick -n abc1234
```

DevOps Context: Cherry-pick a hotfix from `main` to a `release` branch without merging everything.

5.6 Amending the Last Commit

Oops, typo in the commit message:

```
$ git commit --amend -m "Correct message"
```

Forgot to add a file:

```
$ git add forgotten-file.txt  
$ git commit --amend --no-edit
```

DevOps Context: Only amend unpushed commits. Amending pushed commits rewrites public history.

Part 6: Collaboration — Conflicts, Stash & Rebase

6.1 Merge Conflicts

What Causes a Conflict?

When two branches change the **same line** of the **same file**, Git doesn't know which version to keep.

Example conflict markers:


```
<<<<<<< HEAD
This is what YOUR branch has
=====
This is what THE OTHER branch has
>>>>>>> other-branch-name
```

How to Resolve

1. Find conflicts:

```
bash $ git status
```

Look for: "Unmerged paths"

2. Open the file and edit it. Remove the `<<<<<<<`, `=====`, and `>>>>>>>` markers.

3. Stage the resolved file:

```
bash $ git add filename.txt
```

4. Complete the merge:

```
bash $ git commit -m "Resolve merge conflict in filename.txt"
```

Search for conflict markers:

```
$ grep -r "<<<<<<<" .
```

Abort a merge in progress:

```
$ git merge --abort
```

DevOps Context: Conflicts are normal. Communicate with teammates about who owns which files.

6.2 Git Stash

Temporarily save work without committing.

Stash current changes:

```
$ git stash
```

Expected Output:

```
Saved working directory and index state ...
```

Stash with a message:

```
$ git stash push -m "WIP: updating terraform config"
```

List stashes:

```
$ git stash list
```

Expected Output:

```
stash@{0}: WIP on main: abc1234 Add feature  
stash@{1}: On main: WIP terraform migration
```

Apply and remove latest stash:

```
$ git stash pop
```

Apply but keep stash:

```
$ git stash apply
```

Apply specific stash:

```
$ git stash apply stash@{1}
```

Drop a stash:

```
$ git stash drop stash@{0}
```

Clear all stashes:

```
$ git stash clear
```

DevOps Context: *Stash when someone says "Production is down, fix it NOW" but you're in the middle of a refactor.*

6.3 Rebasing

Rebase replays your commits on top of another branch. It creates a cleaner, linear history.

Basic rebase:

```
# On your feature branch
$ git switch feature-name
$ git rebase main
```

Interactive rebase (rewrite history):

```
# Rebase the last 3 commits
$ git rebase -i HEAD~3
```

An editor opens. You can:

- `pick` — keep commit (default)
- `reword` — edit commit message
- `squash` — combine with previous commit
- `drop` — remove commit
- `fixup` — combine and discard message

Continue rebase after resolving conflicts:

```
$ git rebase --continue
```

Abort rebase:

```
$ git rebase --abort
```

DevOps Context: *Rebase feature branches before opening a PR. This eliminates unnecessary merge commits. **NEVER rebase shared branches like `main`.***

6.4 Interactive Rebase Example

Clean up messy commits before a PR:

```
$ git rebase -i HEAD~4
```

Editor shows:

```
pick abc1234 Fix typo
pick def5678 Add config
pick ghi901 Add tests
pick jkl234 Update docs
```

Edit to:

```
reword abc1234 Fix typo
squash def5678 Add config
pick ghi901 Add tests
fixup jkl234 Update docs
```

Result: Clean commits with good messages.

Part 7: DevOps Branching Patterns

7.1 Feature Branch Workflow

Best for: Small teams, startups, most modern DevOps.

```
main (always deployable)
├── feature/login
├── feature/dashboard
└── bugfix/memory-leak
```

Rules:

- `main` is sacred — always works
- All work happens on feature branches
- Every merge goes through PR + code review

7.2 GitFlow

Best for: Large teams with scheduled releases.

```
main          (production)
develop       (integration)
├── feature/...
└── feature/...
hotfix/...    (emergency fixes for production)
release/...   (release preparation)
```

Rules:

- `main` = production only
- `develop` = integration branch
- Features branch from `develop`
- Releases branch from `develop`, merge to `main`
- Hotfixes branch from `main`, merge back to both

7.3 Trunk-Based Development

Best for: CI/CD-heavy teams deploying multiple times daily.

```
main (trunk)
├── short-lived feature branches (hours, not days)
```

Rules:

- Everyone commits to `main` or very short-lived branches
- Feature flags hide incomplete work
- Requires robust CI

7.4 Forking Workflow

Best for: Open source projects.

1. Fork the official repo to your GitHub account
 2. Clone YOUR fork locally
 3. Push branches to YOUR fork
 4. Open PR from your fork → official repo
-
-

Part 8: CI/CD with GitHub Actions

8.1 What is GitHub Actions?

GitHub Actions is a built-in CI/CD platform. It runs code (workflows) in response to Git events (push, PR, schedule).

Analogy: A robot watches your mailbox. Every time you put a letter in, the robot opens it, checks the spelling, rewrites it if needed, and approves or rejects it.

8.2 Workflow Structure

Workflow files live in `.github/workflows/` and use YAML.

```
name: CI Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run tests
        run: make test
```

Key Components:

Component	Description
name	Display name in GitHub UI
on	Events that trigger the workflow
jobs	Groups of steps
runs-on	Type of virtual machine
steps	Individual commands
uses	Reusable action from GitHub Marketplace
run	Shell command

8.3 Common Triggers

```
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  schedule:
    - cron: '0 0 * * 0' # Every Sunday at midnight
  workflow_dispatch: # Manual trigger button
```

8.4 Complete DevOps Workflow Example

```
name: Deploy to Production

on:
  push:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm ci

      - name: Run linter
        run: npm run lint

      - name: Run tests
        run: npm test

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Deploy to server
        run: |
          echo "Deploying to production..."
          # Your deploy scripts here
```

8.5 Branch Protection with Required Status Checks

1. GitHub repo → **Settings** → **Branches**
2. Add rule for `main`
3. Enable:
 - Require pull request before merging

- Require status checks to pass
- Select your workflow name (e.g., `CI Pipeline`)
- Require code review

8.6 Secrets and Environment Variables

Store sensitive data in GitHub settings, NOT in code.

1. Repo → **Settings** → **Secrets and variables** → **Actions**
2. Click **New repository secret**
3. Name: `PROD_API_KEY`
4. Value: Your actual secret

Access in workflow:

```
- name: Deploy
  run: |
    curl -H "Authorization: Bearer ${ secrets.PROD_API_KEY }" ...
```

8.7 Caching for Speed

Cache dependencies to speed up workflows:

```
- name: Cache node modules
  uses: actions/cache@v3
  with:
    path: ~/.npm
    key: ${ runner.os }-node-${ hashFiles('**/package-lock.json') }
```

Part 9: Release Management — Tags & Releases

9.1 Tags

Tags mark specific commits as releases.

Lightweight tag (just a name):

```
$ git tag v1.0.0
```

Annotated tag (with message):

```
$ git tag -a v1.0.0 -m "Release version 1.0.0"
```

List tags:

```
$ git tag
```

Push a single tag:

```
$ git push origin v1.0.0
```

Push all tags:

```
$ git push origin --tags
```

Delete a local tag:

```
$ git tag -d v1.0.0
```

Delete a remote tag:

```
$ git push --delete origin v1.0.0
```

9.2 Semantic Versioning

Format: `MAJOR.MINOR.PATCH`

Version	Meaning
1.0.0	First stable release
1.1.0	New features, backward compatible
1.1.1	Bug fixes only
2.0.0	Breaking changes

9.3 Creating a GitHub Release

1. Go to repo on GitHub → **Releases** (right sidebar)
2. Click **Draft a new release**
3. Choose a tag (or create new)
4. Add release title and notes
5. Attach binaries (optional)
6. Click **Publish release**

9.4 Release Branch Strategy

```
# Create release branch
$ git switch -c release/v2.1.0

# Fix any last-minute issues
$ git commit -m "Bump version to 2.1.0"

# Merge to main and tag
$ git switch main
$ git merge release/v2.1.0
$ git tag -a v2.1.0 -m "Release 2.1.0"
$ git push origin main --tags

# Merge back to develop
$ git switch develop
$ git merge release/v2.1.0
```

Part 10: Advanced Topics

10.1 Git Hooks

Scripts that run automatically on Git events.

Available hooks:

- `pre-commit` — Run before each commit
- `prepare-commit-msg` — Modify commit message

- `post-merge` — Run after merge
- `pre-push` — Run before push

Example: Pre-commit hook to run linter

```
# Create the hook
$ cat > .git/hooks/pre-commit << 'EOF'
#!/bin/bash
echo "Running linter..."
make lint || exit 1
EOF

$ chmod +x .git/hooks/pre-commit
```

Note: Hooks in `.git/hooks` are not shared. Use [Husky](#) or [pre-commit](#) for team-wide hooks.

10.2 Submodules

Include another repo inside your repo.

Add a submodule:

```
$ git submodule add git@github.com:user/shared-scripts.git scripts
```

Clone with submodules:

```
$ git clone --recurse-submodules git@github.com:user/repo.git
```

Update submodules:

```
$ git submodule update --remote
```

10.3 Worktrees

Work on multiple branches simultaneously without cloning multiple times.

```
# Create a new directory for feature branch work
$ git worktree add ../my-project-feature feature-branch

# Now you have two directories, each on a different branch
$ cd ../my-project-feature
$ git branch
# Shows: * feature-branch
```

List worktrees:

```
$ git worktree list
```

Remove a worktree:

```
$ git worktree remove ../my-project-feature
```

10.4 Bisect (Find the Bug)

Binary search through history to find which commit introduced a bug.

```
# Start bisect
$ git bisect start

# Mark current (bad) commit
$ git bisect bad

# Mark last known good commit
$ git bisect good v1.0.0

# Git checks out a middle commit
# Test it, then mark good or bad:
$ git bisect good    # OR git bisect bad

# Repeat until Git identifies the first bad commit

# Reset when done
$ git bisect reset
```

Part 11: Troubleshooting Guide

"fatal: not a git repository"

Cause: You ran a Git command outside a repository.

Fix:

```
$ cd /path/to/your/repo
# OR
$ git init # If you haven't initialized yet
```

"Permission denied (publickey)"

Cause: SSH key not set up or not added to GitHub.

Fix:

1. Generate SSH key: `ssh-keygen -t ed25519 -C "email"`
2. Add to GitHub: Settings → SSH keys
3. Test: `ssh -T git@github.com`

"Merge conflict"

Fix:

```
# Find conflicted files
$ git status

# Open each file, remove <<<<<< ===== >>>>>> markers
# Edit to desired result

# Stage resolved files
$ git add .

# Complete merge
$ git commit -m "Resolve merge conflicts"
```

"Your branch is ahead of 'origin/main' by 3 commits"

Fix:

```
$ git push
```

"Your branch and 'origin/main' have diverged"

Fix:

```
# Option 1: Merge (preserves branch structure)
$ git pull

# Option 2: Rebase (cleaner history)
$ git pull --rebase
```

"The requested URL returned error: 403"

Cause: Wrong permissions or using HTTPS without token.

Fix: Switch to SSH:

```
$ git remote set-url origin git@github.com:USER/REPO.git
```

"Failed to push some refs"

Cause: Remote has commits you don't have locally.

Fix:

```
$ git pull
# Resolve any conflicts
$ git push
```

"I committed to the wrong branch"

Fix:

```
# Create the correct branch (includes the commit)
$ git switch -c correct-branch

# Go back to original branch and remove commit
$ git switch wrong-branch
$ git reset --soft HEAD~1
```

"I committed a secret/password"

Fix:

```
# If NOT pushed yet
$ git reset --soft HEAD~1

# If ALREADY pushed (complex – requires history rewrite)
# Use GitHub's secret scanning or BFG Repo-Cleaner
# Rotate the secret immediately regardless
```

ALSO:

```
$ git rm --cached secret-file
$ git commit -m "Remove secret"
$ echo "secret-file" >> .gitignore
$ git add .gitignore
$ git commit -m "Add secret to .gitignore"
```

"git log shows a mess of merge commits"

Fix: Use rebase before merging:

```
$ git switch feature
$ git rebase main
$ git switch main
$ git merge feature
```

Workflow fails with "Permission denied"

Cause: GitHub Actions token lacks permissions.

Fix: Add permissions to workflow:

```
permissions:
  contents: write
  pull-requests: read
```

Part 12: Command Cheat Sheet

Daily Commands

Command	Description
<code>git status</code>	Check what's changed
<code>git add filename</code>	Stage a file
<code>git add .</code>	Stage all changes
<code>git commit -m "msg"</code>	Commit staged changes
<code>git push</code>	Upload commits
<code>git pull</code>	Download and merge latest
<code>git log --oneline</code>	View compact history
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes

Branch Commands

Command	Description
<code>git branch</code>	List branches
<code>git branch name</code>	Create branch
<code>git switch name</code>	Switch branch
<code>git switch -c name</code>	Create and switch
<code>git merge name</code>	Merge branch into current
<code>git branch -d name</code>	Delete merged branch
<code>git branch -D name</code>	Force delete branch

History & Undo

Command	Description
<code>git log --oneline</code>	Compact history
<code>git show COMMIT</code>	Show commit details
<code>git blame FILE</code>	Who edited each line
<code>git revert COMMIT</code>	Undo commit safely
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes
<code>git reset --hard HEAD~1</code>	Delete last commit AND changes
<code>git reflog</code>	View all actions
<code>git cherry-pick COMMIT</code>	Apply specific commit
<code>git commit --amend</code>	Edit last commit

Stashing

Command	Description
<code>git stash</code>	Save current work
<code>git stash push -m "note"</code>	Save with message
<code>git stash list</code>	List stashes
<code>git stash pop</code>	Apply and remove stash
<code>git stash apply</code>	Apply but keep stash
<code>git stash drop</code>	Delete stash
<code>git stash clear</code>	Delete all stashes

Rebasing

Command	Description
<code>git rebase main</code>	Replay commits on main
<code>git rebase -i HEAD~3</code>	Interactive rebase
<code>git rebase --continue</code>	Continue after conflict
<code>git rebase --abort</code>	Cancel rebase

Remote

Command	Description
<code>git remote -v</code>	Show remotes
<code>git remote add NAME URL</code>	Add remote
<code>git fetch</code>	Download from remote
<code>git pull</code>	Fetch + merge
<code>git push -u origin BRANCH</code>	Push and set upstream
<code>git clone URL</code>	Copy repository

Tags & Releases

Command	Description
<code>git tag</code>	List tags
<code>git tag v1.0.0</code>	Create lightweight tag
<code>git tag -a v1.0.0 -m "msg"</code>	Create annotated tag
<code>git push origin v1.0.0</code>	Push tag
<code>git push origin --tags</code>	Push all tags

Glossary: From 5-Year-Old to Engineer

Kid-Friendly	Professional	Real Definition
Magic Camera	Version Control	System that saves and restores file history
Photo	Commit	A permanent snapshot of files
Prep Box	Staging Area	Where you prepare files for a commit
Photo Album	Repository	All commits, branches, and history
School Library	Remote (GitHub)	Server copy of your repo
Bookmarked Path	Branch	An independent line of development
Combine Paths	Merge	Join two branches together
Permission Slip	Pull Request	Formal request to merge into main
Robot	CI/CD	Automated testing and deployment
Recipe	YAML Workflow	Instructions for the robot

This manual is a living document. Git changes. Best practices evolve. Keep learning, commit often, and may your deploys always be green.

Git & GitHub for DevOps Complete Curriculum

Approach 1: Linear Lecture + Lab (5 Sessions)

Beginner-friendly | 1 Hour per Session | Theory First, Then Practice

Instructor Guide

Philosophy

This curriculum uses a **traditional classroom structure**: present theory first, then immediately apply it. Each session is modular — if a student misses Session 2, they can still follow Session 3 because concepts are reviewed.

Magic Rule: Every term is introduced with an analogy before the technical definition.

Pacing:

- 20 min: Lecture (slides + analogies)
- 25 min: Guided lab (instructor demos, students follow)
- 10 min: Independent exercise (students work alone)
- 5 min: Review + Q&A

Student prerequisites: None.

Required environment: Same as all approaches — see Environment Setup below.

Before You Start

Students must complete the Environment Setup section before Session 1. Reserve 30 minutes for this, or assign as pre-class homework.

Environment Setup Guide

Same setup for all approaches. Do this before Day 1.

Step 1: Check Your Operating System

If you see...	You have
<code>C:\</code> drive or Windows logo	Windows
Finder or <code>~/</code>	Mac
<code>apt-get</code> or <code>dnf</code>	Linux

Step 2: Install Git

Environment: Windows

1. Visit `https://git-scm.com/download/win`
2. Download and run the installer
3. Accept all defaults (click Next repeatedly)
4. Open **Git Bash** from the Start menu
5. Verify:

```
$ git --version
```

Expected Output:

```
git version 2.43.0.windows.1
```

(Version number may differ — that's okay)

Environment: Mac

1. Open **Terminal** (Cmd+Space, type "Terminal")
2. Run:

```
$ git --version
```

Expected Output:

```
git version 2.39.3
```

If prompted to install Developer Tools, click **Install**.

Environment: Linux (Ubuntu/Debian)

```
$ sudo apt update && sudo apt install git -y  
$ git --version
```

Expected Output:

```
git version 2.34.1
```

Environment: Linux (Fedora/CentOS/RHEL)

```
$ sudo dnf install git -y  
$ git --version
```

Step 3: Configure Git Identity

Environment: Any terminal (Git Bash / Terminal)

```
$ git config --global user.name "Your Full Name"  
$ git config --global user.email "youremail@example.com"
```

Analogy: Git needs to know who you are, like a library card needs your name.

Verify:

```
$ git config --global user.name  
$ git config --global user.email
```


Step 4: Create GitHub Account

Environment: Web Browser

1. Go to `https://github.com`
2. Click **Sign up**
3. Use the same email as Step 3
4. Complete verification

Step 5: Set Up SSH Authentication

Environment: Any terminal

```
$ ssh-keygen -t ed25519 -C "youremail@example.com"
```

Press Enter three times to accept defaults and skip passphrase.

Copy the public key:

Mac:

```
$ pbcopy < ~/.ssh/id_ed25519.pub
```

Windows (Git Bash):

```
$ cat ~/.ssh/id_ed25519.pub | clip
```

Linux:

```
$ cat ~/.ssh/id_ed25519.pub
```

(Select and copy the output manually)

Add to GitHub:

1. Go to `https://github.com` → **Settings**
2. **SSH and GPG keys** → **New SSH key**
3. Title: "My laptop"
4. Paste the key
5. Click **Add SSH key**

Test:

```
$ ssh -T git@github.com
```

Type `yes` , press Enter.

Expected Output:

```
Hi yourusername! You've successfully authenticated...
```

Step 6: Install Text Editor

Install VS Code from <https://code.visualstudio.com> . For this course, any text editor is acceptable.

Session 1: What is Git? Your First Repository

Learning Objectives

- Define version control using an analogy
- Initialize a Git repository
- Understand the staging area model
- Create commits with meaningful messages

Lecture: Theory (20 minutes)

Slide 1: The Chaos of No Version Control

Have you ever named a file:

```
report_final.doc  
report_final_v2.doc  
report_final_v2_ACTUAL.doc  
report_final_v2_ACTUAL_FOR_REAL.doc
```

This is **manual version control**. It works until you have:

- 50 files

- 3 people editing simultaneously
- A deadline in 2 hours

Slide 2: The Analogy — The Magic Camera

Analogy: *Git is like a magic camera. You work on your drawing, and when you reach a good stopping point, you take a photo. If you mess up later, you can say "Take me back to Photo #3!" and everything is restored.*

In Git:

- The **drawing** = your project files
- The **photo** = a commit
- The **photo album** = the repository
- The **camera itself** = the `.git` folder

Slide 3: The Staging Area Model

Git does not take photos automatically. You must:

1. **Create/edit files** (Working Directory — your messy desk)
2. **Put files in the box** (Staging Area — your prep zone)
3. **Take the photo** (Commit — permanent snapshot)

This three-stage model is the most important concept in Git.

Slide 4: Why "Staging"?

Imagine taking a family photo. You want everyone looking good, right? You don't just snap the camera when people are blinking. You:

1. Tell everyone to get ready (add to staging)
2. Check that everyone is smiling (`git status`)
3. Actually take the photo (`git commit`)

Slide 5: Commits Need Messages

Every photo needs a label:

- ☐ Bad label: "stuff", "changes", "asdf"
- ☐ Good label: "Add contact form", "Fix navigation CSS"

Think of it as writing on the back of the photo: "This was taken at Grandma's birthday."

Guided Lab: Your First Repository (25 minutes)

Environment: Terminal

Create a project folder:

```
$ cd ~  
$ mkdir devops-course  
$ cd devops-course
```

Check Git status before initialization:

```
$ git status
```

Expected Output:

```
fatal: not a git repository...
```

Initialize Git:

```
$ git init
```

Expected Output:

```
Initialized empty Git repository in .../.git/
```

Check status again:

```
$ git status
```

Expected Output:

```
On branch main  
No commits yet  
nothing to commit
```

Create your first file:

```
$ echo "Welcome to DevOps!" > readme.txt  
$ cat readme.txt
```

Check Git status:

```
$ git status
```

Expected Output:

```
Untracked files:  
  readme.txt
```

Stage the file:

```
$ git add readme.txt
```

Check status:

```
$ git status
```

Expected Output:

```
Changes to be committed:  
  new file:   readme.txt
```

Commit:

```
$ git commit -m "Add welcome message"
```

Expected Output:

```
[main (root-commit) abc1234] Add welcome message  
1 file changed, 1 insertion(+)
```

View history:

```
$ git log
```

Expected Output:

```
commit abc1234...
Author: Your Name <...>
Date: ...

    Add welcome message
```

Make another change:

```
$ echo "This course covers Git and GitHub." >> readme.txt
$ git status
```

Expected Output:

```
modified:   readme.txt
```

Stage and commit:

```
$ git add readme.txt
$ git commit -m "Add course description"
```

View compact log:

```
$ git log --oneline
```

Expected Output:

```
def5678 Add course description
abc1234 Add welcome message
```

Independent Exercise (10 minutes)

Create `tools.txt` with the text `Git, GitHub, CI/CD` . Stage and commit it.

▮ **Solution**

```
$ echo "Git, GitHub, CI/CD" > tools.txt
$ git add tools.txt
$ git commit -m "Add tools list"
```

Verify:

```
$ git log --oneline
```

****Expected Output:****

```
1234abc Add tools list
def5678 Add course description
abc1234 Add welcome message
```

Review (5 minutes)

Key questions:

1. What does `git init` do?
 2. What is the staging area?
 3. What is the difference between `git add` and `git commit` ?
 4. Why should commit messages be descriptive?
-
-

Session 2: Branching and Merging

Learning Objectives

- Explain branching with a "parallel universe" analogy
- Create and switch branches
- Merge branches back to main
- Understand why branching matters in team environments

Lecture: Theory (20 minutes)

Slide 1: The Problem with Working on Main

If three developers edit the same file simultaneously:

- Developer A rewrites the header
- Developer B adds a footer
- Developer C deletes the sidebar

They all save to `main` at the same time. Result: **Chaos**.

Slide 2: The Parallel Universe Analogy

Analogy: A branch is a parallel universe. In Universe A, you're adding a contact form. In Universe B, your teammate is redesigning the homepage. Neither universe affects the other. When both are ready, you merge the universes together.

Slide 3: Branch Naming Conventions

Good Name	Why
<code>feature-contact-form</code>	Clear what it does
<code>fix-broken-link</code>	Clear what it fixes
<code>update-about-page</code>	Clear what it updates

Bad Name	Why
<code>branch1</code>	Meaningless
<code>temp</code>	Will you remember what this was in 2 weeks?
<code>asdf</code>	Seriously?

Slide 4: Fast-Forward vs. Merge Commit

- **Fast-forward:** The main branch hasn't changed since you branched. Git just moves the pointer forward.

- **Merge commit:** The main branch has new commits. Git creates a new commit that combines both branches.

Guided Lab: Branching Workflow (25 minutes)

Environment: Terminal (in `~/devops-course`)

Verify you're on main:

```
$ git branch
```

Expected Output:

```
* main
```

Create a feature branch:

```
$ git branch feature-about  
$ git branch
```

Expected Output:

```
feature-about  
* main
```

Note the `*` is still on `main` . Switch:

```
$ git switch feature-about
```

Expected Output:

```
Switched to branch 'feature-about'
```

Verify:

```
$ git branch
```

Expected Output:

```
* feature-about  
main
```

Create a new file on the branch:

```
$ echo "About this course: learn Git from scratch." > about.txt  
$ git add about.txt  
$ git commit -m "Add about page"
```

Verify commit is on this branch:

```
$ git log --oneline
```

Check that main is unchanged:

```
$ git switch main  
$ ls
```

Expected Output: (no `about.txt`)

```
readme.txt  tools.txt
```

Switch back and merge:

```
$ git switch feature-about  
$ git switch main  
$ git merge feature-about
```

Expected Output:

```
Fast-forward  
about.txt | 1 +
```

Verify main now has the file:

```
$ ls
```

Expected Output:

```
about.txt  readme.txt  tools.txt
```

Delete the branch:

```
$ git branch -d feature-about
```

Check log:

```
$ git log --oneline
```

Independent Exercise (10 minutes)

Create a branch `feature-contact`. Add `contact.txt` with `Email: team@devops.com`. Commit, merge to `main`, delete the branch.

Solution

```
$ git branch feature-contact
$ git switch feature-contact
$ echo "Email: team@devops.com" > contact.txt
$ git add contact.txt
$ git commit -m "Add contact page"
$ git switch main
$ git merge feature-contact
$ git branch -d feature-contact
```

Verify:

```
$ ls
$ git log --oneline
```

Review (5 minutes)

1. Why do we use branches instead of editing `main` directly?
2. What does `git switch` do?
3. What is a fast-forward merge?
4. Why delete branches after merging?

Session 3: GitHub, Remotes, and Pull Requests

Learning Objectives

- Explain what a remote repository is
- Connect local repo to GitHub
- Push and pull code
- Create and merge Pull Requests

Lecture: Theory (20 minutes)

Slide 1: Local vs. Remote

- **Local repository:** Lives on your computer (your bedroom desk)
- **Remote repository:** Lives on GitHub (the school library)

In DevOps, the remote is the "source of truth" — the official version everyone trusts.

Slide 2: Why GitHub?

GitHub adds to Git:

- A web interface for viewing code
- Pull Requests (code review)
- Issues (bug tracking)
- Actions (automation/CI)
- Team collaboration tools

Slide 3: The Workflow

```
Local:    [create branch] → [make changes] → [commit]
                                     ↓
Remote:           [push]
                                     ↓
GitHub:  [open Pull Request] → [review] → [merge]
                                     ↓
Local:           [pull]
```

Slide 4: Pull Requests Explained

Analogy: A Pull Request is like a permission slip. You say: "I want to add my homework to the class folder. Please check it first." The teacher reviews it, and if it's correct, signs it (approves). Then it gets filed (merged).

In real DevOps, NO ONE pushes directly to `main`. All changes go through PR + code review.

Guided Lab: GitHub Workflow (25 minutes)

Environment: Browser + Terminal

Part A: Create GitHub Repository

1. Go to `https://github.com`
2. Click + → **New repository**
3. Name: `devops-course`
4. Description: "DevOps course project"
5. Public
6. Uncheck "Add a README file"
7. Click **Create repository**

Part B: Push Local Code

Environment: Terminal (in `~/devops-course`)

```
$ git remote add origin git@github.com:YOURUSERNAME/devops-course.git
$ git remote -v
```

Expected Output:

```
origin git@github.com:../devops-course.git (fetch)
origin git@github.com:../devops-course.git (push)
```

```
$ git push -u origin main
```

Expected Output:

```
* [new branch]      main -> main
```

Refresh GitHub. Your files are there!

Part C: Branch, Push, and PR

Create and switch to a branch:

```
$ git branch feature-faq
$ git switch feature-faq
```

Create and commit:

```
$ echo "Q: What is DevOps?" > faq.txt
$ echo "A: It's the practice of combining dev and ops." >> faq.txt
$ git add faq.txt
$ git commit -m "Add FAQ page"
```

Push branch:

```
$ git push -u origin feature-faq
```

Environment: Browser

1. On GitHub, click **"Compare & pull request"**
2. Title: Add FAQ page
3. Description:

- Added FAQ file - No breaking changes

4. Click **Create pull request**
5. Click **Files changed** to review
6. Click **Review changes** → **Approve** → **Submit review**
7. Click **Merge pull request** → **Confirm merge**
8. Click **Delete branch**

Part D: Pull Changes Locally

Environment: Terminal

```
$ git switch main  
$ git pull
```

Expected Output:

```
From github.com:.../devops-course  
... -> main  
Fast-forward  
faq.txt | 2 ++
```

Verify:

```
$ ls
```

Delete local branch:

```
$ git branch -d feature-faq
```

Independent Exercise (10 minutes)

Create a PR that adds `services.txt` with three services listed. Follow the full PR workflow.

□ Solution

```
$ git branch feature-services
$ git switch feature-services
$ cat > services.txt << 'EOF'
- Web Hosting
- CI/CD Pipelines
- Cloud Consulting
EOF
$ git add services.txt
$ git commit -m "Add services page"
$ git push -u origin feature-services
```

****On GitHub:**** - Create PR, review, merge, delete branch ****Back in terminal:****

```
$ git switch main
$ git pull
$ git branch -d feature-services
$ ls
```

Review (5 minutes)

1. What is a remote?
 2. What does `git push` do?
 3. What does `git pull` do?
 4. Why do we use Pull Requests instead of pushing directly?
-
-

Session 4: Undoing Changes and Handling Conflicts

Learning Objectives

- View commit history efficiently
- Revert commits safely
- Resolve merge conflicts
- Use git stash for temporary storage

Lecture: Theory (20 minutes)

Slide 1: The Philosophy of Undoing

Git gives you many ways to go back. Choosing the right one matters:

Situation	Right Tool	Why
Bad edit, not committed	<code>git restore</code>	Cleans working directory
Wrong file staged	<code>git restore --staged</code>	Removes from staging
Bad commit, local only	<code>git reset --soft HEAD~1</code>	Removes last commit
Bad commit, already pushed	<code>git revert</code>	Creates undo commit safely

Rule: *Never rewrite history that others have seen.*

Slide 2: Revert vs. Reset

- **Reset:** Erases commits (dangerous on shared history)
- **Revert:** Adds a new commit that undoes an old one (always safe)

In professional DevOps, you almost always use **revert**.

Slide 3: What is a Merge Conflict?

When two branches change the same line, Git says: "I don't know which version you want. You must tell me manually."

Analogy: Two people edit the same sentence in a shared document. The document can't decide which spelling to keep. A human must choose.

Slide 4: Git Stash

When you need to switch contexts quickly but aren't ready to commit:

1. **Stash** your current work (put it in a backpack)

2. Do the urgent thing
3. **Pop** the stash (unpack your backpack)

Guided Lab Part 1: Reverting (12 minutes)

Check current history:

```
$ cd ~/devops-course  
$ git log --oneline
```

Create a bad commit:

```
$ echo "WRONG CONTENT" > readme.txt  
$ git add readme.txt  
$ git commit -m "Accidentally overwrite readme"
```

Find the bad commit ID:

```
$ git log --oneline
```

Example Output:

```
bad9999 Accidentally overwrite readme  
... (older commits)
```

Revert it:

```
$ git revert bad9999
```

An editor opens. Save and exit:

- **Nano:** Ctrl+O, Enter, Ctrl+X
- **Vim:** Esc, `:wq`, Enter

Expected Output:

```
[main fix1234] Revert "Accidentally overwrite readme"
```

Verify the file:

```
$ cat readme.txt
```

Expected: Original content is restored.

Check log:

```
$ git log --oneline
```

Notice the bad commit is still there, plus the revert commit.

Push to GitHub:

```
$ git push
```

Guided Lab Part 2: Merge Conflict (13 minutes)

Create a scenario where conflict happens:

```
$ git branch conflict-demo  
$ git switch conflict-demo
```

Change a line:

```
$ echo "Branch version of readme" > readme.txt  
$ git add readme.txt  
$ git commit -m "Update readme on branch"
```

Switch to main and make a different change:

```
$ git switch main  
$ echo "Main version of readme" > readme.txt  
$ git add readme.txt  
$ git commit -m "Update readme on main"
```

Try to merge:

```
$ git merge conflict-demo
```

Expected Output:

```
CONFLICT (content): Merge conflict in readme.txt
Automatic merge failed
```

View the conflict:

```
$ cat readme.txt
```

Expected Output:

```
<<<<<<< HEAD
Main version of readme
=====
Branch version of readme
>>>>>>> conflict-demo
```

Resolve it by editing:

```
$ echo "Combined version of readme" > readme.txt
```

Mark resolved and commit:

```
$ git add readme.txt
$ git commit -m "Resolve readme conflict"
```

Independent Exercise (10 minutes)

Introduce a conflict on `about.txt` between `main` and a test branch. Resolve it by keeping the best parts of both versions.

□ Solution

```
$ git branch test-conflict
$ git switch test-conflict
$ echo "About: This is the test branch version." > about.txt
$ git add about.txt
$ git commit -m "Update about on test branch"

$ git switch main
$ echo "About: This is the main version." > about.txt
$ git add about.txt
$ git commit -m "Update about on main"

$ git merge test-conflict
# (conflict!)

$ echo "About: This course teaches Git. Test branch + main combined." > about.txt
$ git add about.txt
$ git commit -m "Resolve about conflict"
```

Review (5 minutes)

1. Why is `git revert` safer than `git reset`?
 2. What do conflict markers look like?
 3. After fixing a conflict, what two commands must you run?
 4. When would you use `git stash`?
-
-

Session 5: DevOps Patterns and GitHub Actions (CI/CD)

Learning Objectives

- Explain CI/CD with an analogy
- Write a GitHub Actions workflow in YAML
- Understand triggers and runners
- Set up basic branch protection

Lecture: Theory (20 minutes)

Slide 1: What is CI/CD?

- **CI (Continuous Integration):** Automatically test code on every change
- **CD (Continuous Deployment):** Automatically deploy if tests pass

Analogy: A robot watches the mailbox. Every time a letter arrives, it:

1. Opens the letter
2. Checks spelling
3. Makes a copy for records
4. Delivers it if everything looks good

Slide 2: Why DevOps Needs CI/CD

Without CI/CD	With CI/CD
Developer pushes code	Developer pushes code
Someone manually tests	Robot automatically tests
Days later, bug found	Minutes later, pass/fail known
Manual deployment	Automatic deployment

Slide 3: GitHub Actions Basics

- **Workflow:** A YAML file in `.github/workflows/`
- **Trigger:** Events like `push`, `pull_request`, `schedule`
- **Job:** A group of steps
- **Step:** A single command or action
- **Runner:** A VM that executes your workflow

Slide 4: Branch Protection

Branch protection rules:

- Require PR before merging
- Require status checks (CI must pass)
- Require code review

This prevents direct pushes to `main` and enforces quality.

Guided Lab: Build a CI Pipeline (25 minutes)

Environment: Terminal

Create workflow directory:

```
$ cd ~/devops-course  
$ mkdir -p .github/workflows
```

Write a workflow:

```
$ cat > .github/workflows/ci.yml << 'EOF'  
name: CI Check  
  
on:  
  push:  
    branches: [ main ]  
  pull_request:  
    branches: [ main ]  
  
jobs:  
  verify:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - name: Check files exist  
        run: |  
          test -f readme.txt && echo "readme.txt OK"  
          test -f about.txt && echo "about.txt OK"  
          test -f contact.txt && echo "contact.txt OK"  
          echo "All checks passed!"  
  
EOF
```

Commit and push:

```
$ git add .github/workflows/ci.yml  
$ git commit -m "Add CI workflow"  
$ git push
```

Environment: Browser

1. Go to **Actions** tab
2. Watch the workflow run
3. Click the run to see green checkmarks

Test Failure Scenario

Create a branch:

```
$ git branch break-build
$ git switch break-build
$ rm contact.txt
$ git add contact.txt
$ git commit -m "Remove contact (should fail CI)"
$ git push -u origin break-build
```

On GitHub:

- Open the PR
- CI will fail with red X
- Show the failed step
- Close the PR without merging

Set Branch Protection

1. Repo → **Settings** → **Branches**
2. Add rule: `main`
3. Check:
 - Require pull request
 - Require status checks: `CI Check`
4. Click **Create**

Verify by trying direct push:

```
$ git switch main
$ echo "test" >> readme.txt
$ git add readme.txt
$ git commit -m "Test direct push"
$ git push
```

Expected: Error — protected branch.

Undo the test commit:

```
$ git reset --soft HEAD~1
$ git restore --staged readme.txt
$ git checkout -- readme.txt
```


Independent Exercise (10 minutes)

Add a new status check step for `faq.txt` . Push and verify it runs on the Actions tab.

Solution

Edit `.github/workflows/ci.yml` to add:

```
- name: Check FAQ
  run: test -f faq.txt && echo "faq.txt OK"
```

```
$ git add .github/workflows/ci.yml
$ git commit -m "Add FAQ check to CI"
$ git push
```

Watch it on the Actions tab on GitHub.

Review and Capstone (5 minutes)

Students demonstrate:

1. All files in GitHub repo
2. At least one merged PR
3. Passing CI workflow
4. Branch protection enabled

Appendix A: Command Quick Reference

Command	Description
git init	Initialize a repository
git status	Check current state
git add <file>	Stage a file
git commit -m "msg"	Commit staged files
git log --oneline	Compact history
git branch <name>	Create branch
git switch <name>	Change branch
git merge <name>	Merge branch into current
git remote add origin <url>	Add remote
git push -u origin <branch>	Push branch to remote
git pull	Fetch and merge from remote
git revert <commit>	Undo a commit safely
git stash	Save temporary work
git stash pop	Restore stashed work

Appendix B: Session Timing

Session	Lect	Lab	Exercise	Review	Total
1	20	25	10	5	60
2	20	25	10	5	60
3	20	25	10	5	60
4	20	25	10	5	60
5	20	25	10	5	60

End of Approach 1: Linear Lecture + Lab

Git & GitHub for DevOps Complete Curriculum

Approach 2: Project-Driven ("Build-Along")

Beginner-friendly | 5 Sessions | 1 Continuous Project | Learn by Building

Instructor Guide

Philosophy

Students learn by **building one real project from start to finish**. Every session adds a feature to the project. By Session 5, they have a fully version-controlled website with CI/CD. The project IS the curriculum.

Magic Rule: No abstract examples — every command serves the project.

Pacing:

- 10 min: Project status check + plan for today
- 15 min: New concept (introduced because the project needs it)
- 25 min: Hands-on building (instructor demos, students follow)
- 10 min: Extend the feature (student challenge)

Student prerequisites: None.

The Project: `my-devops-portfolio` — a simple portfolio website. By Session 5 it has:

- Multiple pages (Home, About, Projects, Contact)
- Branch-based feature development
- GitHub collaboration workflow
- Automated CI testing
- Branch protection

Before You Start

Complete Environment Setup before Session 1.

Environment Setup Guide

Identical for all three approaches.

Step 1: Install Git

Environment: Windows

1. <https://git-scm.com/download/win>
2. Install with all defaults
3. Open **Git Bash**
4. Verify:

```
$ git --version
```

Expected Output:

```
git version 2.43.0.windows.1
```

Environment: Mac

```
$ git --version
```

If not installed, macOS will prompt to install Developer Tools.

Environment: Linux (Ubuntu/Debian)

```
$ sudo apt update && sudo apt install git -y  
$ git --version
```

Environment: Linux (Fedora)

```
$ sudo dnf install git -y  
$ git --version
```

Step 2: Configure Git

Environment: Any terminal

```
$ git config --global user.name "Your Full Name"  
$ git config --global user.email "youremail@example.com"
```

Step 3: Create GitHub Account

Environment: Browser

1. `https://github.com`
2. Sign up with the same email
3. Verify email

Step 4: SSH Keys

Environment: Terminal

```
$ ssh-keygen -t ed25519 -C "youremail@example.com"
```

Press Enter three times.

Copy key:

Mac: `pbcopy < ~/.ssh/id_ed25519.pub`

Windows: `cat ~/.ssh/id_ed25519.pub | clip`

Linux: `cat ~/.ssh/id_ed25519.pub` (copy manually)

Add to GitHub: Settings → SSH and GPG keys → New SSH key

Test:

```
$ ssh -T git@github.com
```

Type `yes`.

Expected Output:

```
Hi yourusername! You've successfully authenticated...
```

Step 5: Install VS Code

Download from <https://code.visualstudio.com>.

The Project: `my-devops-portfolio`

We will build a personal portfolio website across 5 sessions. Here's the roadmap:

Session	Feature Added	Git Skill
1	Home page	Init, add, commit, status, log
2	About + Projects pages	Branching, merging
3	Push to GitHub + Contact page	Remotes, push, pull, PRs
4	Fix a bug + Style updates	Revert, resolve conflicts
5	Auto-deploy check	CI/CD with GitHub Actions

Session 1: Foundation — Home Page and First Commits

The Goal Today

Create the project folder, initialize Git, and build the Home page. Save your progress using commits.

Theory: What We Need to Know (15 minutes)

The Analogy — Building a House

Analogy: Building a project without Git is like building a house without taking photos. If the wall collapses, you have no idea what it looked like before.

Git is your construction photographer. Every time you finish a room, you take a photo (commit). If the roof leaks, you can look at the photos and figure out when it started.

The Three Stages

Think of Git like a restaurant kitchen:

1. **Working Directory** = The prep table (you're cutting vegetables)
2. **Staging Area** = The plate ready to be served (chef has arranged it)
3. **Commit** = The photo for the menu (permanent record)

You don't take menu photos of half-chopped vegetables. You stage first, then commit.

Practical Build: Home Page (35 minutes)

Environment: Terminal

Create the project:

```
$ cd ~  
$ mkdir my-devops-portfolio  
$ cd my-devops-portfolio
```

Initialize Git:

```
$ git init
```

Expected Output:

```
Initialized empty Git repository in .../.git/
```

Create the homepage:

```
$ cat > home.txt << 'EOF'
=====
  MY DEVOPS PORTFOLIO
=====

Hi, I'm [Your Name]!

I'm learning DevOps and this is my first Git project.

Visit the About page to learn more.
EOF
```

Check status:

```
$ git status
```

Expected Output:

```
Untracked files:
  home.txt
```

Stage it:

```
$ git add home.txt
```

Check status again:

```
$ git status
```

Expected Output:

```
Changes to be committed:
  new file:   home.txt
```

Commit:

```
$ git commit -m "Add homepage with welcome message"
```

Expected Output:

```
[main (root-commit) abc1234] Add homepage with welcome message  
1 file changed, 10 insertions(+)
```

Check history:

```
$ git log --oneline
```

Expected Output:

```
abc1234 Add homepage with welcome message
```

Now add a navigation section:

Environment: Text Editor

Open `home.txt` and add to the bottom:

```
---  
Navigation: Home | About | Projects | Contact  
---
```

Or via terminal:

```
$ cat >> home.txt << 'EOF'  
  
---  
Navigation: Home | About | Projects | Contact  
---  
EOF
```

Check status:

```
$ git status
```

Expected Output:

```
modified:   home.txt
```

Stage and commit:

```
$ git add home.txt
$ git commit -m "Add navigation section to homepage"
```

View log:

```
$ git log --oneline
```

Expected Output:

```
def5678 Add navigation section to homepage
abc1234 Add homepage with welcome message
```

Student Extension Challenge (10 minutes)

Add a footer to `home.txt`: `Built with Git`. Stage and commit it.

Solution

```
$ echo "" >> home.txt
$ echo "Built with Git" >> home.txt
$ git add home.txt
$ git commit -m "Add footer to homepage"
```

Verify:

```
$ git log --oneline
```

****Expected:**** A third commit.

Session 2: Two New Pages — Branching for Features

The Goal Today

Add two new pages (About and Projects) using feature branches. Keep `main` clean and stable.

Theory: Why Branches Matter (15 minutes)

The Parallel Universe Analogy

Analogy: You're writing a book. The published version is `main`. You want to try a new chapter, but you're not sure if readers will like it. So you photocopy the book, write the new chapter in the copy, and show it to friends. If they like it, you paste it into the real book. If they hate it, you throw away the copy.

The photocopy = a branch. The real book = `main`.

Professional Context

In DevOps:

- `main` is **sacred** — always deployable
- Feature branches are **disposable** — experiment freely
- You merge only when code is tested and reviewed

Practical Build: About and Projects Pages (35 minutes)

Environment: Terminal (in `~/my-devops-portfolio`)

Ensure you're on `main`:

```
$ git branch
```

Expected Output:

```
* main
```

Feature 1: About Page

Create branch:

```
$ git branch feature-about-page  
$ git switch feature-about-page
```

Check:

```
$ git branch
```

Expected Output:

```
* feature-about-page  
  main
```

Build the About page:

```
$ cat > about.txt << 'EOF'  
=====
```

ABOUT ME

```
=====
```

Name: [Your Name]
Role: DevOps Student
Goal: To master Git, GitHub, and CI/CD

This portfolio tracks my learning journey.
EOF

Stage and commit:

```
$ git add about.txt  
$ git commit -m "Add about page with bio"
```

Feature 2: Projects Page

Create a second branch (from `main` , not from the about branch):

First, go back to main:

```
$ git switch main
```

Create projects branch:

```
$ git branch feature-projects-page  
$ git switch feature-projects-page
```

Build the Projects page:

```
$ cat > projects.txt << 'EOF'  
=====
```

MY PROJECTS

```
=====
```

1. DevOps Portfolio (this website!)

- Tools: Git, GitHub, CI/CD
- Status: In progress

More projects coming soon...

EOF

Stage and commit:

```
$ git add projects.txt  
$ git commit -m "Add projects page"
```

Verify main is still clean:

```
$ git switch main  
$ ls
```

Expected Output:

```
home.txt
```

(Only `home.txt` — the new pages are on branches, not main!)

Merge About Page First

```
$ git merge feature-about-page
```

Expected Output:

```
Fast-forward  
about.txt | 10 ++++++++
```

Merge Projects Page

```
$ git merge feature-projects-page
```

Expected Output:

```
Merge made by the 'ort' strategy.  
projects.txt | 11 ++++++++
```

Note: This may be a merge commit (not fast-forward) because `main` now has the About page.

Verify all files:

```
$ ls
```

Expected Output:

```
about.txt  home.txt  projects.txt
```

View the merge commit in log:

```
$ git log --oneline --graph
```

Expected Output:


```
* merge123 Merge branch 'feature-projects-page'
|\
| * proj456 Add projects page
* | about789 Add about page with bio
|/
* def5678 Add navigation section
```

Clean up branches:

```
$ git branch -d feature-about-page
$ git branch -d feature-projects-page
```

Student Extension Challenge (10 minutes)

Create a `skills.txt` file on a new branch `feature-skills` with 3 skills listed. Merge it into `main` and delete the branch.

▮ Solution

```
$ git branch feature-skills
$ git switch feature-skills
$ cat > skills.txt << 'EOF'
Skills:
- Git version control
- GitHub collaboration
- CI/CD pipelines
EOF
$ git add skills.txt
$ git commit -m "Add skills page"
$ git switch main
$ git merge feature-skills
$ git branch -d feature-skills
```

Verify:

```
$ ls
$ git log --oneline
```

Session 3: Going Online — GitHub and Pull Requests

The Goal Today

Push the portfolio to GitHub. Add a Contact page through a proper Pull Request workflow.

Theory: Sharing with the World (15 minutes)

The Library Analogy

Analogy: Your local Git repo is like a diary on your desk. GitHub is like a library where you publish your diary so others can read it. You push (upload) your finished chapters. You pull (download) edits from your editor.

Why PRs?

You wouldn't publish a book without an editor reading it first. A Pull Request is like handing your chapter to the editor and saying "Is this good enough for the book?"

In DevOps:

- Pushing directly to `main` = editing the published book while it's in stores
- Opening a PR = submitting to the editor first

Practical Build: Contact Page via PR (35 minutes)

Environment: Browser + Terminal

Part 1: Create GitHub Repo

1. Go to `https://github.com`
2. Click + → **New repository**
3. Name: `my-devops-portfolio`
4. Public, no README
5. Click **Create**

Part 2: Push Local Code

Environment: Terminal

```
$ git remote add origin git@github.com:YOURUSERNAME/my-devops-portfolio.git  
$ git push -u origin main
```

Expected Output:

```
* [new branch]      main -> main
```

Refresh GitHub — your files are there!

Part 3: Contact Page via PR

Create branch:

```
$ git branch feature-contact-page  
$ git switch feature-contact-page
```

Build the page:

```
$ cat > contact.txt << 'EOF'  
=====
```

CONTACT

```
=====
```

Email: your.email@example.com
GitHub: github.com/YOURUSERNAME

Let's connect!
EOF

Stage and commit:

```
$ git add contact.txt  
$ git commit -m "Add contact page"
```

Push:

```
$ git push -u origin feature-contact-page
```

On GitHub:

1. Click "**Compare & pull request**"
2. Title: `Add contact page`
3. Description:
 - Added contact information - Follows existing page style
4. **Create pull request**
5. **Files changed** → review
6. **Review changes** → **Approve** → **Submit**
7. **Merge pull request** → **Confirm**
8. **Delete branch**

Part 4: Sync Local Main

Environment: Terminal

```
$ git switch main  
$ git pull
```

Expected Output:

```
From github.com:.../my-devops-portfolio  
...  
contact.txt | 10 ++++++++
```

Verify:

```
$ ls
```

Expected Output:

```
about.txt  contact.txt  home.txt  projects.txt  skills.txt
```

Clean up:

```
$ git branch -d feature-contact-page
```

Student Extension Challenge (10 minutes)

Update `projects.txt` to include a "Future Goals" section. Do it through a PR workflow: branch, edit, commit, push, PR, merge, pull.

Solution

```
$ git branch update-projects
$ git switch update-projects
```

****Edit `projects.txt` to add:****

```
Future Goals:
- Complete CI/CD certification
- Contribute to open source
- Build a cloud infrastructure project
```

```
$ git add projects.txt
$ git commit -m "Add future goals to projects page"
$ git push -u origin update-projects
```

****On GitHub:**** Create PR, review, merge, delete branch. ****Back in terminal:****

```
$ git switch main
$ git pull
$ git branch -d update-projects
```

Session 4: Fixing Mistakes and Handling The Unexpected

The Goal Today

Introduce a "bug" (wrong content), find it in history, revert it safely. Also handle a merge conflict when two branches change the same file.

Theory: When Things Go Wrong (15 minutes)

The Time Machine

`git log --oneline` is your time machine. It shows every snapshot. Find the bad one, then use `git revert` to undo it.

Key rule: *We never erase history. We add to it. Revert creates a NEW commit that undoes an old one. This keeps the audit trail.*

Conflicts Are Normal

When two branches touch the same line, Git asks a human to choose. It's not scary — it's just Git saying "I need help deciding."

Practical Build: Bug Fix + Conflict Resolution (35 minutes)

Part 1: The Bug

Accidentally overwrite the homepage:

Environment: Terminal (on `main`)

```
$ echo "Oops, this is bad content." > home.txt
$ git add home.txt
$ git commit -m "Redesign homepage (broken)"
```

Check log:

```
$ git log --oneline
```

Find the bad commit ID (top one).

Revert it:

```
$ git revert HEAD
```

(HEAD means "the most recent commit on current branch")

An editor opens. Save and exit.

Expected Output:

```
[main revert123] Revert "Redesign homepage (broken)"
```

Verify the file:

```
$ cat home.txt
```

The original content is restored!

Push to GitHub:

```
$ git push
```

Part 2: The Merge Conflict

Create two branches that both edit `about.txt` :

```
$ git branch update-about-role  
$ git switch update-about-role
```

Change the role line:

```
$ cat > about.txt << 'EOF'
=====
  ABOUT ME
=====

Name: [Your Name]
Role: DevOps Engineer (updated role)
Goal: To master Git, GitHub, and CI/CD

This portfolio tracks my learning journey.
EOF
$ git add about.txt
$ git commit -m "Update role on about page"
```

Now create another branch from `main` :

```
$ git switch main
$ git branch update-about-goal
$ git switch update-about-goal
```

Change the goal line:

```
$ cat > about.txt << 'EOF'
=====
  ABOUT ME
=====

Name: [Your Name]
Role: DevOps Student
Goal: To become a Senior DevOps Engineer

This portfolio tracks my learning journey.
EOF
$ git add about.txt
$ git commit -m "Update goal on about page"
```

Merge the first branch:

```
$ git switch main
$ git merge update-about-role
```

This works fine. Now merge the second:


```
$ git merge update-about-goal
```

Expected Output:

```
CONFLICT (content): Merge conflict in about.txt
Automatic merge failed
```

View the file:

```
$ cat about.txt
```

Expected Output:

```
<<<<<<< HEAD
Role: DevOps Engineer (updated role)
=====
Goal: To become a Senior DevOps Engineer
>>>>>>> update-about-goal
```

(Note: the actual conflict may include more context depending on Git version)

Resolve by combining both changes:

```
$ cat > about.txt << 'EOF'
=====
  ABOUT ME
=====

Name: [Your Name]
Role: DevOps Engineer (updated role)
Goal: To become a Senior DevOps Engineer

This portfolio tracks my learning journey.
EOF
```

Mark as resolved:

```
$ git add about.txt
$ git commit -m "Merge about updates and resolve conflict"
```

Clean up:

```
$ git branch -d update-about-role  
$ git branch -d update-about-goal  
$ git push
```

Student Extension Challenge (10 minutes)

Create a conflict on `contact.txt` between `main` and a new branch. Resolve it by keeping a merged version with both email and GitHub info enhanced.

▮ Solution

```
$ git branch contact-update-1
$ git switch contact-update-1
$ echo "Email: my.email@example.com (preferred)" > contact.txt
$ git add contact.txt
$ git commit -m "Update email format"

$ git switch main
$ git branch contact-update-2
$ git switch contact-update-2
$ echo "GitHub: github.com/YOURUSERNAME (check out my repos!)" > contact.txt
$ git add contact.txt
$ git commit -m "Enhance GitHub link"

$ git switch main
$ git merge contact-update-1
$ git merge contact-update-2
# (conflict)

# Resolve
$ cat > contact.txt << 'EOF'
=====
CONTACT
=====

Email: my.email@example.com (preferred)
GitHub: github.com/YOURUSERNAME (check out my repos!)

Let's connect!
EOF

$ git add contact.txt
$ git commit -m "Merge contact updates"
$ git branch -d contact-update-1
$ git branch -d contact-update-2
```

Session 5: Robots Do the Work — CI/CD with GitHub Actions

The Goal Today

Add a GitHub Actions workflow that automatically checks our portfolio whenever someone pushes code. Lock down `main` with branch protection.

Theory: Automation is DevOps (15 minutes)

The Robot Chef

Analogy: Imagine a restaurant where every time a chef adds a new dish to the menu, a robot automatically:

- 1. Tastes the dish
- 2. Checks the temperature
- 3. Photographs it for social media
- 4. Either approves it for the menu or throws it away

GitHub Actions is that robot. Every time you push code, it runs tests automatically.

Why This Matters

Manual Process	Automated Process
Developer pushes code	Developer pushes code
Team lead emails: "Did you test?"	Robot tests immediately
Bug found in production	Bug caught before merge
Blame and stress	Confidence and speed

Practical Build: CI Workflow (35 minutes)

Part 1: The Workflow File

Environment: Terminal (on `main`)

```
$ cd ~/my-devops-portfolio  
$ git switch main  
$ mkdir -p .github/workflows
```

Create the workflow:

```

$ cat > .github/workflows/portfolio-ci.yml << 'EOF'
name: Portfolio CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  check-portfolio:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Check required pages exist
        run: |
          for file in home.txt about.txt projects.txt contact.txt; do
            if [ -f "$file" ]; then
              echo "☐ $file found"
            else
              echo "☐ $file MISSING"
              exit 1
            fi
          done

      - name: Check for navigation keywords
        run: |
          if grep -q "Navigation" home.txt; then
            echo "☐ Navigation section found"
          else
            echo "☐ Navigation section missing"
            exit 1
          fi

      - name: Report success
        run: echo "Portfolio validation complete!"
EOF

```

Stage, commit, push:

```

$ git add .github/workflows/portfolio-ci.yml
$ git commit -m "Add CI workflow for portfolio validation"
$ git push

```

Environment: Browser

1. Go to the **Actions** tab on GitHub
2. Watch the workflow run
3. It should be green ☑

Part 2: Test Failure

Create a branch that breaks the build:

```
$ git branch break-nav
$ git switch break-nav
$ cat > home.txt << 'EOF'
=====
MY DEVOPS PORTFOLIO
=====

Navigation removed for testing.
EOF
$ git add home.txt
$ git commit -m "Remove navigation (this should fail CI)"
$ git push -u origin break-nav
```

On GitHub:

- Open the PR
- The workflow runs automatically
- It FAILS ☐ because "Navigation" keyword is missing
- Show students the failed step output
- Close PR without merging

Part 3: Branch Protection

1. GitHub repo → **Settings** → **Branches**
2. Add rule: `main`
3. Check:
 - **Require pull request before merging**
 - **Require status checks to pass**
 - Select `Portfolio CI`
4. **Create**

Verify by trying direct push:

```
$ git switch main
$ echo "Direct push test" >> home.txt
$ git add home.txt
$ git commit -m "Test direct push"
$ git push
```

Expected: Error — branch is protected.

Undo:

```
$ git reset --soft HEAD~1
$ git restore --staged home.txt
$ git checkout -- home.txt
```

Student Extension Challenge (10 minutes)

Add a check to the CI that verifies `skills.txt` exists. Push it and watch it run.

▣ Solution

Edit ``.github/workflows/portfolio-ci.yml`` and add inside the ``run:`` block of "Check required pages exist":

```
- name: Check required pages exist
  run: |
    for file in home.txt about.txt projects.txt contact.txt skills.txt; do
      if [ -f "$file" ]; then
        echo "▣ $file found"
      else
        echo "▣ $file MISSING"
        exit 1
      fi
    done
```

Or via terminal (be careful with indentation):

```
$ git switch main
# edit file manually in a text editor
$ git add .github/workflows/portfolio-ci.yml
$ git commit -m "Extend CI to check skills page"
$ git push
```

Watch it in the Actions tab.

Final Deliverable Checklist

By the end of Session 5, every student should have:

- ☐ A GitHub repo at `github.com/YOURUSERNAME/my-devops-portfolio`
- ☐ At least 5 text files (pages)
- ☐ A commit history with meaningful messages
- ☐ At least 2 merged pull requests
- ☐ A passing GitHub Actions workflow
- ☐ Branch protection enabled on `main`

This repo is their proof of DevOps skills for a job interview!

Appendix: Quick Reference

Command	What It Does
<code>git init</code>	Start version control
<code>git status</code>	See what's changed
<code>git add <file></code>	Stage changes
<code>git commit -m "msg"</code>	Save a snapshot
<code>git log --oneline</code>	View history
<code>git branch <name></code>	Create branch
<code>git switch <name></code>	Change branch
<code>git merge <name></code>	Combine branches
<code>git push -u origin <branch></code>	Upload branch
<code>git pull</code>	Download latest
<code>git revert <commit></code>	Undo safely
<code>git stash / git stash pop</code>	Temporary save

End of Approach 2: Project-Driven Build-Along

Git & GitHub for DevOps Complete Curriculum

Approach 3: Story-Mode ("DevOps Team Onboarding")

Beginner-friendly | 5 Sessions | 1 Hour Each | Theory + Practical Interwoven

Instructor Guide

Philosophy

This curriculum treats learning Git like joining a real DevOps team. Students are not memorizing abstract commands — they are living a story. Every session is a "day at work." This creates emotional context, which is how humans remember things best.

Magic Rule: If a 5-year-old can't understand the analogy, don't use it.

Pacing:

- Each session is exactly 1 hour
- 15 min: Story setup + theory (slides)
- 30 min: Hands-on lab
- 10 min: Practice exercise (student does it alone)
- 5 min: Review + Q&A

Student prerequisites: None. Absolutely none. They don't need to know what a terminal is.

Required environment:

- Computer with internet
- Terminal access (Linux/Mac native, Git Bash on Windows)
- Git installed (see Environment Setup)

- GitHub account (free)
- Text editor (VS Code recommended, but Notepad works for Session 1)

Before You Start

Make students do the "Environment Setup" section on their own BEFORE the first session. It takes 20-30 minutes. Do NOT use class time for this unless they are on locked-down machines.

Environment Setup Guide

Do this before Day 1. Every command below includes the environment where it runs.

Check Your Environment

First, figure out what computer you have. It's like knowing whether you drive a car, truck, or bicycle.

If you see...	You have
A Windows logo or <code>C:\</code> drive	Windows
A Finder or <code>~/</code> path	Mac
A Penguin or <code>apt-get</code>	Linux

Step 1: Install Git

Environment: Windows

1. Go to `https://git-scm.com/download/win`
2. Download the installer
3. Run it. When it asks questions, just click **Next** on everything (defaults are fine for beginners)
4. Open **Git Bash** (search for it in Start menu)
5. Verify: type the command below and press Enter

```
$ git --version
```

Expected Output:

```
git version 2.43.0.windows.1
```

(Your number might be different — that's fine!)

Environment: Mac

1. Open **Terminal** (press Cmd+Space, type `terminal`, press Enter)
2. Type this command:

```
$ git --version
```

Expected Output:

```
git version 2.39.3
```

If it says "command not found" or asks to install "Developer Tools", click **Install** and wait 5 minutes. Then try again.

Environment: Linux (Ubuntu/Debian)

1. Open your terminal (Ctrl+Alt+T usually)
2. Type:

```
$ sudo apt update && sudo apt install git -y
```

1. Verify:

```
$ git --version
```

Expected Output:

```
git version 2.34.1
```

Environment: Linux (Fedora/CentOS/RHEL)

```
$ sudo dnf install git -y  
$ git --version
```

Step 2: Configure Git (Tell Git Who You Are)

Analogy: *Git is like a diary. It needs to know your name so it can write "By [Your Name]" on every page.*

Run these commands in your terminal (Git Bash on Windows, Terminal on Mac/Linux).

Environment: Any terminal (Git Bash / Terminal)

```
$ git config --global user.name "Your Full Name"  
$ git config --global user.email "youremail@example.com"
```

Use the SAME email you will use for GitHub.

Verify it worked:

```
$ git config --global user.name
```

Expected Output:

```
Your Full Name
```

```
$ git config --global user.email
```

Expected Output:

```
youremail@example.com
```

Step 3: Create a GitHub Account

Analogy: *GitHub is like a library where you store your diary so others can read it.*

1. Go to `https://github.com` in your browser
2. Click **Sign up**
3. Use the SAME email you used in Step 2
4. Create a password and username
5. Click **Create account**
6. Check your email for a verification code and enter it

Done! You now have a library card.

Step 4: Set Up SSH Keys (The Secret Handshake)

Analogy: *SSH is like a secret handshake. It proves to GitHub that your computer really belongs to you, so you don't need to type your password every time.*

Environment: Any terminal (Git Bash / Terminal)

```
$ ssh-keygen -t ed25519 -C "youremail@example.com"
```

You will see:

```
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/home/you/.ssh/id_ed25519):
```

Just press **Enter** (accept the default).

Next:

```
Enter passphrase (empty for no passphrase):
```

Press **Enter** again (twice) to skip a password. This is fine for learning.

Expected final output:

```
Your identification has been saved in /home/you/.ssh/id_ed25519  
Your public key has been saved in /home/you/.ssh/id_ed25519.pub
```

Now copy your public key to the clipboard:

Mac:

```
$ pbcopy < ~/.ssh/id_ed25519.pub
```

Windows (Git Bash):

```
$ cat ~/.ssh/id_ed25519.pub | clip
```

Linux:

```
$ cat ~/.ssh/id_ed25519.pub
```

Select the output and copy it manually.

Now add it to GitHub:

1. Go to `https://github.com` in your browser
2. Click your profile picture (top right) → **Settings**
3. In the left sidebar, click **SSH and GPG keys**
4. Click **New SSH key**
5. Title: type "My Laptop"
6. Key: paste what you copied (it starts with `ssh-ed25519`)
7. Click **Add SSH key**

Test it:

```
$ ssh -T git@github.com
```

Expected Output:

```
The authenticity of host 'github.com (...)' can't be established...  
Are you sure you want to continue connecting? (yes/no/[fingerprint])?
```

Type `yes` and press Enter.

Final expected output:

```
Hi yourusername! You've successfully authenticated, but GitHub does not provide shell
```

📦 Environment setup complete!

Step 5: Install a Text Editor

Recommended: VS Code (<https://code.visualstudio.com>)

- Install it from the website
- For beginners, it's friendly and free

Alternative: You can use Notepad (Windows), TextEdit (Mac), or Nano/Gedit (Linux) for this course.

Session 1: "Day 1 at the Company"

Learning the Tools + Your First Repository

Learning Objectives

By the end of this session, students will:

- Understand what version control is, using a simple analogy
- Create their first Git repository
- Make their first commit
- Read Git status like a dashboard

Slide Outline (15 minutes)

Slide 1: Title — "Welcome to TechCorp! You're the new DevOps engineer."

Slide 2: The Problem — "Have you ever saved a file as `report_final.doc` , then `report_final2.doc` , then `report_final_ACTUAL.doc` ?"

Slide 3: The Analogy — "Git is like a time machine for your files. Every time you save a version, Git can take you back to that exact moment."

Slide 4: DevOps Context — "In a company, 10 people edit the same files. Without Git, they'd overwrite each other. With Git, everyone has their own copy and Git merges the changes."

Slide 5: Key Terms (with icons)

- **Repository (Repo):** A project folder that Git watches
- **Commit:** A snapshot of your files at one moment in time
- **Working Directory:** Your normal files (like a messy desk)
- **Staging Area:** A box where you put files before taking a photo of them
- **History:** All the snapshots/commits you've ever made

Slide 6: Today's Mission — "Create your first repo and save your first snapshot."

Theory Script (Read or Present)

"Imagine you're drawing a picture. You draw a sun, then a house, then a tree. Now your little sister draws a monster on it. You cry because your beautiful picture is ruined!"

What if you had a magic camera? Every time you finish something good, you take a photo. If your sister ruins it, you just say 'take me back to photo #2!' and POOF — your picture is safe again.

Git is that magic camera. Every photo is called a **commit**. The folder where you keep all your drawings AND the magic camera is called a **repository** (or repo for short).

Today, you're joining a pretend company called TechCorp. Your boss says: 'Please set up our project folder.' You will create a repository for a simple website."

Practical Lab — "Create Your First Repo" (30 minutes)

Story: "TechCorp's website is just a text file right now. Your job is to put it under version control so your team can track changes."

Step 1: Create a Folder

Environment: Terminal (Git Bash / Terminal)

```
$ cd ~  
$ mkdir techcorp-website  
$ cd techcorp-website
```

Expected Output:

(No output for `mkdir` and `cd` — that's normal!)

Verify where you are:

```
$ pwd
```

Expected Output:

```
/home/you/techcorp-website
```

(Mac/Windows path will look different, but should end with `techcorp-website`)

Step 2: Check Git Status

```
$ git status
```

Expected Output:

```
fatal: not a git repository (or any of the parent directories): .git
```

Explain: "Git is not watching this folder yet. It's like a security guard who hasn't been hired for this building."

Step 3: Initialize the Repository (Hire the Security Guard)

```
$ git init
```

Expected Output:

```
Initialized empty Git repository in /home/you/techcorp-website/.git/
```

Explain: "The `.git` folder is Git's office. It stores every photo/commit. Never delete it!"

Now check status again:

```
$ git status
```

Expected Output:

```
On branch main
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

Step 4: Create a File

Environment: Terminal (or use a text editor if you prefer)

```
$ echo "Welcome to TechCorp!" > index.txt
```

Explain: "The `echo` command writes text into a file. The `>` symbol means 'put this into that file.'"

Check what's inside:

```
$ cat index.txt
```

Expected Output:

```
Welcome to TechCorp!
```

Step 5: Check Git Status Again

```
$ git status
```

Expected Output:

```
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  index.txt
```

Explain: "Git sees the file! But it says 'untracked.' That means Git knows the file exists but hasn't taken a photo of it yet. Imagine the security guard saying 'I see a painting on the floor, but I haven't recorded it in my logbook.'"

Step 6: Add the File to the Staging Area

Analogy: *The staging area is like a box where you put things before you take a photo. You might put a toy in the box, then decide you don't want it in the photo, so you take it out. That's staging.*

```
$ git add index.txt
```

Expected Output:

(No output — silence means success in Git!)

Check status:

```
$ git status
```

Expected Output:

```
On branch main
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   index.txt
```

Explain: "Git now says 'new file: index.txt' under 'Changes to be committed.' That means the file is IN the staging box, ready for the photo."

Step 7: Take the Photo (Make Your First Commit)

```
$ git commit -m "Add homepage with welcome message"
```

Expected Output:

```
[main (root-commit) abc1234] Add homepage with welcome message
1 file changed, 1 insertion(+)
create mode 100644 index.txt
```

Explain what each part means:

- `main` = the default branch name (like the main storyline in a video game)
- `root-commit` = the very first commit (the beginning of history)
- `abc1234` = the commit's unique ID (like a photo number)
- The message in quotes = a note you write on the back of the photo so you remember what changed

Step 8: Check Your History

```
$ git log
```

Expected Output:

```
commit abc1234def5678 (HEAD -> main)
Author: Your Name <youremail@example.com>
Date:   [today's date]
```

```
    Add homepage with welcome message
```

Explain: "This is your time machine! `git log` shows every photo you've ever taken. HEAD → main means 'you are looking at the latest commit on the main branch.'"

Step 9: Change the File and See Git Track It

Let's edit the file to add a new line.

```
$ echo "We build amazing things." >> index.txt
```

Explain: `>>` means "add this to the end of the file." (Two arrows = append. One arrow = overwrite.)

Check the file:

```
$ cat index.txt
```

Expected Output:

```
Welcome to TechCorp!  
We build amazing things.
```

Check Git status:

```
$ git status
```

Expected Output:

```
On branch main  
Changes not staged for commit:  
  (use "git add <file>..." to update what will be committed)  
  (use "git restore <file>..." to discard changes in working directory)  
        modified:   index.txt
```

Explain: "Git sees the file changed! It says 'modified.' But it won't take a photo until we put it back in the staging box."

Stage and commit:

```
$ git add index.txt  
$ git commit -m "Add company description"
```

Expected Output:

```
[main def5678] Add company description  
1 file changed, 1 insertion(+)
```

Check log again:

```
$ git log
```

Expected Output:


```
commit def5678... (HEAD -> main)
Author: Your Name ...
Date: ...
```

Add company description

```
commit abc1234...
Author: Your Name ...
Date: ...
```

Add homepage with welcome message

📌 **Session 1 Complete!** You have a time machine with 2 saved states.

Student Practice Exercise (10 minutes)

Scenario: Create a file called `about.txt` with the text "Founded in 2024." Stage it and commit it with the message "Add about page."

📌 Solution

```
$ echo "Founded in 2024." > about.txt
$ git add about.txt
$ git commit -m "Add about page"
```

****Expected Output:****

```
[main ghi9012] Add about page
1 file changed, 1 insertion(+)
create mode 100644 about.txt
```

Verify:

```
$ git log --oneline
```

****Expected Output:****

```
ghi9012 Add about page
def5678 Add company description
abc1234 Add homepage with welcome message
```

Common Beginner Mistakes

Mistake	Why It Happens	Fix
<code>fatal: not a git repository</code>	Ran <code>git</code> commands outside of the repo	Use <code>cd</code> to navigate into the folder first
Commits but <code>git status</code> still shows changes	Forgot <code>git add</code> before <code>git commit</code>	Stage first, then commit
Typo in commit message	Rushed	It's okay! We'll learn to fix it in Session 2
Can't find <code>.git</code> folder	It's hidden by default	Use <code>ls -la</code> (Mac/Linux) or <code>dir /a</code> (Windows)

Review (5 minutes)

Ask the class:

1. "What is a repository?" (A folder Git watches)
 2. "What is a commit?" (A snapshot/photo of your files)
 3. "What's the staging area?" (A box where we prepare files before committing)
 4. "What do `git add` and `git commit` do together?" (Add = put in box. Commit = take photo)
-
-

Session 2: "Fixing Your First Bug"

Branching + Working on Features Without Breaking Main

Learning Objectives

By the end of this session, students will:

- Understand what a branch is (with a simple analogy)
- Create, switch between, and merge branches
- See how branches prevent breaking the main project

Slide Outline (15 minutes)

Slide 1: Title — "Oh No! The Website is Broken on Main!"

Slide 2: The Problem — "You need to fix a typo, but you don't want to break the live website while you're figuring out the fix."

Slide 3: The Analogy — "Imagine you're reading a choose-your-own-adventure book. The main story is one path. But you can bookmark a page, try a different path, and if it works, go back and rewrite the main story. A branch is a bookmarked alternate path."

Slide 4: DevOps Context — "In real teams, you NEVER edit the `main` branch directly. You create a feature branch, test it, then merge it back. This protects the live code."

Slide 5: Key Terms

- **Branch:** A parallel timeline (like a copy of the story)
- **git branch :** Creates a new timeline
- **git checkout (or git switch):** Jumps to a different timeline
- **Merge:** Combines two timelines back together

Slide 6: Today's Mission — "The boss says the homepage has a typo. Create a branch, fix it, and merge it back."

Theory Script

"In our story, our website is 'live' on the `main` branch. Imagine `main` is the highway that real customers drive on. If you start digging up the highway to fix a pothole, cars will crash!

So what do construction workers do? They close off one lane, work there, and only merge it back into the highway when it's safe.

*In Git, a **branch** is that closed-off lane. You create a branch, do your work safely, test it, and then **merge** it back into `main` .*

The magic is: your `main` branch stays safe the whole time. If you mess up on your branch, just delete it and start over. No customers were harmed."

Practical Lab — "The Typo Fix" (30 minutes)

Story: "A customer emailed: 'Your website says "We build amaizing things" — that's not a word!' Your task: Fix the typo without touching the live site."

Preparation

Make sure you're in your repo from Session 1.

Environment: Terminal (in `~/techcorp-website`)

```
$ cd ~/techcorp-website
```

Step 1: Check What Branch You're On

```
$ git branch
```

Expected Output:

```
* main
```

Explain: "The `*` means 'you are here.' Like a 'You Are Here' dot on a mall map."

Step 2: Create a New Branch

```
$ git branch fix-typo
```

Check branches again:

```
$ git branch
```

Expected Output:

```
fix-typo  
* main
```

Explain: "Notice: we created the lane, but we're still on the highway (`main`). The `*` is still on `main` ."

Step 3: Switch to the New Branch

```
$ git switch fix-typo
```

Expected Output:

```
Switched to branch 'fix-typo'
```

Verify:

```
$ git branch
```

Expected Output:

```
* fix-typo  
main
```

Explain: "The star moved! We're now working on the closed-off lane. Any changes we make here do NOT affect `main` yet."

Step 4: Make the Fix

Introduce a typo first (so we have something to fix):

```
$ echo "We build amaizing things." > index.txt
```

Wait — we already had good text. Let's see the real scenario. First, let's see what we have:

```
$ cat index.txt
```

Let's say it really does have the typo. Fix it:

```
$ echo "Welcome to TechCorp!" > index.txt  
$ echo "We build amazing things." >> index.txt
```

Better way — use a text editor:

Open `index.txt` in your text editor and change "amaizing" to "amazing", then save.

Check what Git sees:

```
$ git status
```

Expected Output:

```
On branch fix-typo
Changes not staged for commit:
  modified:   index.txt
```

Stage and commit:

```
$ git add index.txt
$ git commit -m "Fix typo: amaizing -> amazing"
```

Expected Output:

```
[fix-typo 123def0] Fix typo: amaizing -> amazing
1 file changed, 1 insertion(+), 1 deletion(-)
```

Step 5: Check That Main is Still Safe

```
$ git switch main
```

Look at the file:

```
$ cat index.txt
```

Expected Output: (The OLD version — with the typo still there!)

```
Welcome to TechCorp!
We build amaizing things.
```

Explain: "HALLELUJAH! `main` is safe. Our fix is ONLY on the `fix-typo` branch. The highway is still open."

Switch back:

```
$ git switch fix-typo  
$ cat index.txt
```

Expected Output: (The fixed version)

```
Welcome to TechCorp!  
We build amazing things.
```

Step 6: Merge the Fix Back Into Main

Now that we tested the fix, we merge the lane back into the highway.

```
$ git switch main  
$ git merge fix-typo
```

Expected Output:

```
Updating def5678..123def0  
Fast-forward  
 index.txt | 2 +-  
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Explain: "Fast-forward means Git just moved the `main` pointer forward. No complicated merging needed — it was a straight line."

Verify:

```
$ cat index.txt
```

Expected Output:

```
Welcome to TechCorp!  
We build amazing things.
```

Step 7: Delete the Branch (Cleanup)

The lane has been merged into the highway. We don't need the lane anymore.

```
$ git branch -d fix-typo
```

Expected Output:

```
Deleted branch fix-typo (was 123def0).
```

Verify:

```
$ git branch
```

Expected Output:

```
* main
```

Check the log:

```
$ git log --oneline
```

Expected Output:

```
123def0 Fix typo: amaizing -> amazing  
def5678 Add company description  
abc1234 Add homepage with welcome message
```

☐ **Bug fixed, main is safe, history is clean!**

Student Practice Exercise (10 minutes)

Scenario: The boss wants a "Contact Us" page. Create a branch called `add-contact`. Add a file `contact.txt` with the text `Email: hello@techcorp.com`. Commit it. Switch back to `main` and verify `contact.txt` does NOT exist there. Then merge `add-contact` into `main` and verify it DOES exist. Finally, delete the branch.

☐ **Solution**


```
# Create and switch to branch
$ git branch add-contact
$ git switch add-contact

# Create file and commit
$ echo "Email: hello@techcorp.com" > contact.txt
$ git add contact.txt
$ git commit -m "Add contact page"

# Verify main doesn't have it
$ git switch main
$ ls
```

****Expected Output:**** `about.txt index.txt` (note: NO `contact.txt`)

```
# Merge the branch
$ git merge add-contact
$ ls
```

****Expected Output:**** `about.txt contact.txt index.txt`

```
# Cleanup
$ git branch -d add-contact
```

Common Beginner Mistakes

Mistake	Why It Happens	Fix
Edited files on <code>main</code> directly	Forgot to create/switch branch	Create branch from the fixed commit, or learn <code>git stash</code> (Session 4)
<code>git merge</code> says "Already up to date"	Tried to merge in the wrong direction	Make sure you're ON <code>main</code> and merging the feature branch INTO it
Can't delete branch	Used lowercase <code>-d</code> when branch isn't merged	Use <code>-D</code> (uppercase) to force delete, or merge first

Session 3: "Submitting Your Work"

GitHub Remotes, Push, Pull, and Pull Requests

Learning Objectives

By the end of this session, students will:

- Understand what a "remote" is (with analogy)
- Push their local repo to GitHub
- Create and merge a Pull Request
- Understand the code review workflow

Slide Outline (15 minutes)

Slide 1: Title — "Time to Show the Team!"

Slide 2: The Problem — "Your code is on your laptop. Your boss can't see it. Your teammates can't see it. How do you share?"

Slide 3: The Analogy — "Your laptop is like your bedroom desk. GitHub is like the school library. You do your homework at your desk, then you bring it to the library so your teacher and classmates can see it too. Pushing is 'bringing it to the library.' Pulling is 'getting the latest homework from the library.'"

Slide 4: DevOps Context — "In real teams, you never push directly to `main` on GitHub. You push your branch, open a Pull Request (PR), ask someone to review it, THEN merge it. This is how bugs get caught before they reach customers."

Slide 5: Key Terms

- **Remote:** A copy of your repo on another computer (usually GitHub)
- **Push:** Send your commits to the remote
- **Pull:** Download commits from the remote
- **Clone:** Make a copy of a remote repo on your computer
- **Pull Request (PR):** Asking permission to merge your branch into main

Slide 6: Today's Mission — "Put your repo on GitHub. Create a feature branch. Open a PR. Get it approved (by yourself for practice) and merge it."

Theory Script

"So far, all your code is on your laptop. Imagine doing homework on your bedroom desk and never showing anyone. That's fine for practice, but in a real job, your TEAM needs to see your work."

GitHub is like a library. You push (upload) your code there. Your team can review it, comment on it, and suggest changes.

But here's the rule at TechCorp: Nobody is allowed to change the library's master copy directly. You must open a **Pull Request** — it's like filling out a form that says 'I think my homework is correct. Please review it before putting it in the official folder.'

Another engineer reviews your code, says 'Looks good!' and clicks merge. This is called **code review** and it's one of the most important practices in DevOps. It catches mistakes before they reach real customers."

Practical Lab — "First PR" (30 minutes)

Story: "The boss wants an FAQ page. You must add it, push it to GitHub, open a Pull Request, review it yourself, and merge it."

Step 1: Create a GitHub Repository

Environment: Browser (GitHub.com)

1. Go to `https://github.com` and log in
2. Click the + button (top right) → **New repository**
3. Repository name: `techcorp-website`
4. Description: "TechCorp official website"
5. Choose **Public** (Private costs money for teams)
6. **DO NOT** check "Initialize this repository with a README"
7. Click **Create repository**

You will see a page with instructions. Look for the section titled **"...or push an existing repository from the command line."**

Step 2: Connect Your Local Repo to GitHub

Environment: Terminal (in `~/techcorp-website`)

```
$ git remote add origin git@github.com:YOURUSERNAME/techcorp-website.git
```

Replace YOURUSERNAME with your actual GitHub username.

Verify it worked:

```
$ git remote -v
```

Expected Output:

```
origin  git@github.com:YOURUSERNAME/techcorp-website.git (fetch)
origin  git@github.com:YOURUSERNAME/techcorp-website.git (push)
```

Explain: " `origin` is just a nickname for the remote server. Like saving a phone number as 'Mom' instead of typing the full number every time."

Step 3: Push Main to GitHub

```
$ git push -u origin main
```

Expected Output:

```
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
... (more lines)
To github.com:YOURUSERNAME/techcorp-website.git
 * [new branch]      main -> main
```

Explain: "Your code is now on GitHub! `-u origin main` tells Git: 'Remember that from now on, when I say `git push` on the `main` branch, push to `origin`.'"

Refresh your GitHub page in the browser. You should see:

- `index.txt` file
- `about.txt` file
- Your commit history

Step 4: Create a Feature Branch for the FAQ Page

```
$ git branch add-faq  
$ git switch add-faq
```

Step 5: Create the FAQ File

```
$ echo "Q: What does TechCorp do?" > faq.txt  
$ echo "A: We build amazing DevOps tools." >> faq.txt
```

Check what you made:

```
$ cat faq.txt
```

Expected Output:

```
Q: What does TechCorp do?  
A: We build amazing DevOps tools.
```

Stage and commit:

```
$ git add faq.txt  
$ git commit -m "Add FAQ page"
```

Step 6: Push the Branch to GitHub

```
$ git push -u origin add-faq
```

Expected Output:

```
* [new branch]      add-faq -> add-faq
```

Refresh GitHub. You will now see a yellow banner: "**add-faq had recent pushes. Compare & pull request.**"

Step 7: Open a Pull Request

Environment: Browser (GitHub.com)

1. Click the yellow banner's **"Compare & pull request"** button
2. Title: `Add FAQ page`
3. Description:
 - Added FAQ page with common questions - No breaking changes
4. Click **Create pull request**

Step 8: Review and Approve the PR

Since you're practicing alone, you will review your own PR.

1. Scroll down to see the "Files changed" tab or click it
2. You should see `faq.txt` with a green **+** showing the new content
3. Click the **Review changes** button (top right of the file)
4. Select **Approve**
5. Click **Submit review**

Step 9: Merge the PR

1. Click the **Conversation** tab
2. Click the green **Merge pull request** button
3. Click **Confirm merge**
4. Click **Delete branch** (cleanup)

Step 10: Pull the Changes to Your Local Main

"Your local `main` doesn't know about the merge yet. You need to pull (download) it."

Environment: Terminal

```
$ git switch main  
$ git pull
```

Expected Output:

```
remote: Enumerating objects: ...
Unpacking objects: 100% ...
From github.com:YOURUSERNAME/techcorp-website
    123def0..789abcd  main      -> origin/main
Updating 123def0..789abcd
Fast-forward
  faq.txt | 2 ++
  1 file changed, 2 insertions(+)
```

Verify locally:

```
$ ls
```

Expected Output:

```
about.txt  faq.txt  index.txt
```

```
$ git log --oneline
```

Expected Output: (includes the merge)

```
789abcd Add FAQ page
... (older commits)
```

Delete the local branch:

```
$ git branch -d add-faq
```

🎉 **Your first Pull Request is complete!**

Student Practice Exercise (10 minutes)

Scenario: Add a `services.txt` file with the text `Web Hosting, CI/CD, Cloud Consulting`. Do it all through a PR workflow:

1. Create branch `add-services`
2. Create file, commit
3. Push branch, open PR, merge on GitHub
4. Pull to local main
5. Delete branch everywhere

▮ Solution

```
# Local work
$ git branch add-services
$ git switch add-services
$ echo "Web Hosting, CI/CD, Cloud Consulting" > services.txt
$ git add services.txt
$ git commit -m "Add services page"

# Push and PR
$ git push -u origin add-services
```

****Then on GitHub:**** - Click "Compare & pull request" - Create PR, review, merge, delete branch ****Back in terminal:****

```
$ git switch main
$ git pull
$ git branch -d add-services
```

Verify:

```
$ ls
$ cat services.txt
```

Common Beginner Mistakes

Mistake	Why It Happens	Fix
<code>Permission denied</code> on push	SSH not set up correctly	Redo the SSH setup in Environment Setup
Pushed to <code>main</code> directly	Forgot to create a branch	Talk about branch protection rules (Session 5)
PR shows way more changes than expected	Branched from wrong place	Always branch from the latest <code>main</code>
<code>git pull</code> shows merge conflicts	Someone else changed the same file	Session 4 covers this

Session 4: "The Deploy Broke!"

Undoing, Reverting, and Handling Conflicts

Learning Objectives

By the end of this session, students will:

- Use `git log` effectively to find old commits
- Revert a bad commit safely
- Resolve a simple merge conflict
- Understand `git stash` for emergency context switches

Slide Outline (15 minutes)

Slide 1: Title — "3 AM Page: Production is Down!"

Slide 2: The Problem — "You just deployed code, and the website shows a white screen. The boss is calling. You need to undo the change FAST."

Slide 3: The Analogy — "You have a time machine (`git log`). But you can't erase the past — people might have seen it. Instead, you add a NEW commit that undoes the bad one. It's like saying 'Forget my last move' in a board game."

Slide 4: DevOps Context — "In DevOps, you rarely 'delete' history because audit trails matter. Instead, you **revert** — create a new commit that undoes an old one. This is safe, traceable, and professional."

Slide 5: Key Terms

- `git log --oneline` : A compact history view
- `git revert` : Create a new commit that undoes an old commit
- **Merge Conflict**: Two people changed the same line; Git needs a human to decide which version to keep
- `git stash` : Put your current work in a backpack so you can do something else

Slide 6: Today's Mission — "Accidentally break something, find the bad commit, revert it, and handle a merge conflict."

Theory Script

"At 3 AM, your phone rings. The website is broken. The last deploy was 20 minutes ago. You know it worked before that."

Here's what you DON'T do: panic and delete files. Here's what you DO:

- 1. Look at the history (`git log --oneline`)*
- 2. Find the bad commit*
- 3. Revert it (`git revert BAD_COMMIT`)*
- 4. Push the revert to GitHub*

Why revert instead of delete? Because in a real company, there might be laws (compliance) saying you can't erase history. Also, other developers might have already built on top of your bad commit. Reverting creates a NEW commit that undoes the bad one — history stays intact, but the code goes back to working.

*Also today: **merge conflicts**. Imagine you and your teammate both edited the same sentence in a document. Git sees both versions and says 'I don't know which one you want.' YOU have to tell it. We'll practice that."*

Practical Lab Part 1 — "The Revert" (15 minutes)

Story: "Someone (you!) accidentally deleted the 'Contact Us' info and committed it. Revert that commit to restore it."

Step 1: Break Something on Purpose

Environment: Terminal (in repo, on `main`)

```
$ cd ~/techcorp-website
$ git switch main
```

Delete a line from `index.txt` and overwrite with bad info:

```
$ echo "Oops, we deleted everything!" > index.txt
$ git add index.txt
$ git commit -m "Refactor homepage"
```

Check log:

```
$ git log --oneline
```

Expected Output: (something like)

```
bad1234 Refactor homepage
789abcd Add FAQ page
123def0 Fix typo: amaizing -> amazing
```

Step 2: Find the Bad Commit

```
$ git log --oneline
```

Expected Output:

```
bad1234 Refactor homepage
789abcd Add FAQ page
```

"The top one (`bad1234`) broke everything! We need to undo it."

Step 3: Revert the Bad Commit

```
$ git revert bad1234
```

An editor will open (Nano or Vim). It shows the default commit message:

```
Revert "Refactor homepage"

This reverts commit bad1234...
```

Save and exit:

- **Nano:** Press Ctrl+O, then Enter, then Ctrl+X
- **Vim:** Press Esc, type `:wq` , press Enter

Expected Output:

```
[main undo5678] Revert "Refactor homepage"  
1 file changed, 1 insertion(+), 1 deletion(-)
```

Verify:

```
$ cat index.txt
```

Expected Output: (The GOOD version is back!)

```
Welcome to TechCorp!  
We build amazing things.
```

Check the log:

```
$ git log --oneline
```

Expected Output:

```
undo5678 Revert "Refactor homepage"  
bad1234 Refactor homepage  
789abcd Add FAQ page
```

Explain: "Notice: the bad commit (`bad1234`) is STILL in history. We didn't erase it. We ADDED a new commit (`undo5678`) that undoes it. This is the professional way."

Push to GitHub:

```
$ git push
```

Practical Lab Part 2 — "The Merge Conflict" (15 minutes)

Story: "You and a teammate both tried to update the homepage headline. Git can't decide which version is right. You must choose."

Step 1: Simulate Two People Changing the Same File

Create a conflict manually. First, change `index.txt` on `main`:

```
$ echo "Welcome to SUPER TechCorp!" > index.txt
$ git add index.txt
$ git commit -m "Update headline on main"
```

Create a branch from BEFORE that commit:

```
$ git branch conflict-branch HEAD~1
$ git switch conflict-branch
```

Explain: `HEAD~1` means "one commit before the latest." Like saying "the previous photo."

Now make a DIFFERENT change to the same file:

```
$ echo "Welcome to TechCorp Inc." > index.txt
$ git add index.txt
$ git commit -m "Update headline on branch"
```

Step 2: Try to Merge

```
$ git switch main
$ git merge conflict-branch
```

Expected Output:

```
Auto-merging index.txt
CONFLICT (content): Merge conflict in index.txt
Automatic merge failed; fix conflicts and then commit the result.
```

❏ **Conflict!**

Step 3: See What Git Did

```
$ cat index.txt
```

Expected Output:

```
<<<<<<< HEAD
Welcome to SUPER TechCorp!
=====
Welcome to TechCorp Inc.
>>>>>>> conflict-branch
```

Explain the markers:

- <<<<<<< HEAD = "This is what main has"
- ===== = "Separator"
- >>>>>>> conflict-branch = "This is what the branch has"

Step 4: Fix the Conflict

Edit `index.txt` to say what you want. Let's combine them nicely:

```
$ echo "Welcome to SUPER TechCorp Inc.!" > index.txt
```

Important: Remove the conflict markers! If you use `echo` to overwrite, they're automatically gone. If you use a text editor, manually delete `<<<<<<<`, `=====`, and `>>>>>>>` lines.

Step 5: Tell Git It's Fixed

```
$ git add index.txt
$ git commit -m "Merge branch 'conflict-branch' and resolve headline conflict"
```

Expected Output:

```
[main merge9012] Merge branch 'conflict-branch' and resolve headline conflict
```

Verify:

```
$ git log --oneline
$ cat index.txt
```

☐ **Conflict resolved professionally!**

Step 6: Git Stash (Bonus Tool)

```
$ git switch main
```

Make a quick change but don't commit:

```
$ echo "Under construction" >> index.txt
```

Check status:

```
$ git status
```

"Oh no! The boss calls and says fix something on `main` immediately, but you're not done with this change."

Stash it (put it in a backpack):

```
$ git stash
```

Expected Output:

```
Saved working directory and index state ...
```

"Your messy desk is now clean, but your work is saved in a backpack."

Do your urgent fix, then restore your work:

```
$ git stash pop
```

Expected Output:

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
   modified:   index.txt
```

"Your backpack is unpacked and your work is back on the desk!"

Student Practice Exercise (10 minutes)

Scenario:

1. Add an emoji to `about.txt` and commit it on `main`
2. Create branch `better-about` from BEFORE that commit
3. On `better-about`, change the SAME LINE to something different
4. Merge `better-about` into `main` — you'll get a conflict
5. Resolve it by keeping the best parts of both
6. Stash a pretend "work in progress" on `index.txt`, then pop it

▮ Solution

```
# Step 1
$ echo "Founded in 2024! 🦄" > about.txt
$ git add about.txt
$ git commit -m "Add emoji to about"

# Step 2 & 3
$ git branch better-about HEAD~1
$ git switch better-about
$ echo "Founded in 2024 – Excellence since day one." > about.txt
$ git add about.txt
$ git commit -m "Improve about page text"

# Step 4
$ git switch main
$ git merge better-about
# (conflict happens)

# Step 5
$ cat about.txt
# (see conflict markers)
$ echo "Founded in 2024! 🦄 Excellence since day one." > about.txt
$ git add about.txt
$ git commit -m "Resolve about page conflict"

# Step 6
$ echo "WIP: redesigning everything" >> index.txt
$ git stash
$ git stash pop
```


Common Beginner Mistakes

Mistake	Why It Happens	Fix
Edited conflict markers but didn't <code>git add</code>	Thought editing was enough	Must stage (<code>git add</code>) and commit after fixing
Left conflict markers in the file	Didn't clean up completely	Search for <code><<<<<<</code> in the file
<code>git revert</code> on the wrong commit	Didn't read <code>git log --oneline</code> carefully	Use <code>git show COMMIT_ID</code> to preview a commit
<code>git stash</code> and forgot where it went	Stashed too many things	<code>git stash list</code> shows all stashes

Session 5: "Automating Everything"

GitHub Actions: Continuous Integration for DevOps

Learning Objectives

By the end of this session, students will:

- Understand what CI/CD means with a simple analogy
- Create a GitHub Actions workflow file
- See a workflow run automatically on `push`
- Understand how Git events trigger automation

Slide Outline (15 minutes)

Slide 1: Title — "Robots Do the Boring Work"

Slide 2: The Problem — "Every time someone pushes code, the boss wants: 'Did it break anything? Does it build? Can we deploy it?' Manually checking this is exhausting."

Slide 3: The Analogy — "Imagine every time you put a letter in the mailbox, a robot automatically opens it, checks the spelling, rewrites it if needed,

photocopies it, and puts it on everyone's desk. That's CI/CD — robots doing checks every time you change code."

Slide 4: DevOps Context — "CI/CD is the HEART of DevOps. **Continuous Integration** = automatically test code when it's pushed. **Continuous Deployment** = automatically deploy if tests pass. Today we build a simple CI pipeline."

Slide 5: Key Terms

- **CI/CD:** Continuous Integration / Continuous Deployment
- **Workflow:** A recipe that tells GitHub what to do
- **Trigger:** An event that starts the workflow (like `push` , `pull_request`)
- **Runner:** A computer that GitHub rents to run your recipe
- **YAML:** The language used to write the recipe (it's like a shopping list)

Slide 6: Today's Mission — "Build a robot that automatically checks our `index.txt` exists every time we push code."

Theory Script

*"In a real DevOps job, your team pushes code 50 times a day. You can't have a human check every single one. That's where **robots** come in.*

***GitHub Actions** is a robot factory. You write a recipe in a special language called YAML. The recipe says: 'When someone pushes code, do these steps.'*

A typical recipe looks like:

- 1. Get a fresh computer (runner)*
- 2. Install the tools we need*
- 3. Check out our code*
- 4. Run tests (or, for our simple example, verify files exist)*
- 5. Report PASS or FAIL*

If the robot reports FAIL, the developer knows they broke something without anyone else having to tell them. This saves HOURS every day.

YAML looks scary but it's just indented lists. Like a recipe in a cookbook:

Recipe: Chocolate Cake Steps: - Preheat oven to 350 - Mix flour and

`sugar`

That's it. That's YAML."

Practical Lab — "Build Your First Robot" (30 minutes)

Story: "TechCorp wants a robot that automatically checks: 'Does our website have at least index.txt and about.txt?' Every time someone pushes code, the robot runs and reports green (pass) or red (fail)."

Step 1: Create the Workflow File

Environment: Terminal (in repo, on `main`)

```
$ cd ~/techcorp-website  
$ git switch main
```

GitHub Actions looks for workflow files in a special folder: `.github/workflows/`

Create it:

```
$ mkdir -p .github/workflows
```

Create the workflow file:

```
$ cat > .github/workflows/check-files.yml << 'EOF'
name: Check Website Files

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  check-files:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Check required files exist
        run: |
          if [ ! -f "index.txt" ]; then
            echo "ERROR: index.txt is missing!"
            exit 1
          fi
          if [ ! -f "about.txt" ]; then
            echo "ERROR: about.txt is missing!"
            exit 1
          fi
          echo "All required files present!"

EOF
```

Explain what each line does:

- `name:` = What the robot job is called (shows up in GitHub)
- `on:` = When does the robot wake up? (on `push` to `main` or on `pull_request`)
- `jobs:` = The list of tasks
- `runs-on: ubuntu-latest` = "Use a fresh Linux computer"
- `uses: actions/checkout@v4` = "Download our code onto that computer"
- `run: |` = "Run these shell commands"

Step 2: Commit and Push

```
$ git add .github/workflows/check-files.yml
$ git commit -m "Add CI workflow to check required files"
$ git push
```

Step 3: Watch the Robot Work

Environment: Browser (GitHub.com)

1. Go to your repo on GitHub
2. Click the **Actions** tab (next to Pull requests)
3. You should see your workflow running!
4. Click on the yellow dot → it turns green 🟢

If you click inside, you'll see each step:

- Checkout code → 🟢
- Check required files exist → 🟢 with "All required files present!"

Step 4: Break Something and See It Fail

Create a branch and intentionally delete a required file:

Environment: Terminal

```
$ git branch break-ci
$ git switch break-ci
$ git rm about.txt
$ git commit -m "Remove about page (this should fail CI)"
$ git push -u origin break-ci
```

Environment: Browser (GitHub.com)

1. Open the Pull Request for `break-ci`
2. GitHub will run the workflow automatically
3. It will show a red X 🛑 next to the commit
4. Inside Actions, you'll see:

```
ERROR: about.txt is missing!
```

Explain: "The robot caught the mistake BEFORE anyone merged it! This is the power of CI."

Don't merge this PR. Close it (click **Close pull request**) to keep your repo clean.

Step 5: View the Workflow File in Context

Environment: Terminal

Show what's in your `.github/workflows` folder:

```
$ ls .github/workflows/
```

Expected Output:

```
check-files.yml
```

Show all CI workflows in GitHub via command line:

```
$ gh workflow list
```

Expected Output:

NAME	STATE	ID
Check Website Files	active	12345678

(Requires `gh` CLI installed — optional for class, but cool to show.)

Step 6: Branch Protection (The Safety Lock)

Environment: Browser (GitHub.com)

1. Go to your repo → **Settings** tab
2. In the left sidebar, click **Branches**
3. Under "Branch protection rules," click **Add rule**
4. Branch name pattern: `main`
5. Check: **Require a pull request before merging**
6. Check: **Require status checks to pass before merging**
 - Search for and select: `Check Website Files`
7. Click **Create**

Explain: "Now the `main` branch is LOCKED. Nobody can push directly to it. Every change must go through a PR, AND the robot must give a green checkmark. This is how real DevOps teams protect production."

Test it:

```
$ git switch main
$ echo "Direct push test" >> index.txt
$ git add index.txt
$ git commit -m "Try to push directly"
$ git push
```

Expected Output:

```
remote: error: GH006: Protected branch update failed...
remote: error: At least 1 approving review is required...
```

"BLOCKED! The safety lock works."

Revert that commit locally:

```
$ git reset --soft HEAD~1
$ git restore --staged index.txt
$ git checkout -- index.txt
```

(Don't worry about understanding these commands perfectly — just know they undo the last commit and restore the file.)

Student Practice Exercise (10 minutes)

Scenario: Add a second workflow (or extend the first one) that checks `services.txt` also exists. Commit it, push it, and verify it runs in the Actions tab.

▮ Solution

Edit ``.github/workflows/check-files.yml`` and add another check:

```
$ cat >> .github/workflows/check-files.yml << 'EOF'
- name: Check services file
  run: |
    if [ ! -f "services.txt" ]; then
      echo "ERROR: services.txt is missing!"
      exit 1
    fi
    echo "services.txt is present!"
EOF
```

Oops — that appends to the file but may mess up indentation. Better to use a text editor to add this block INSIDE the `steps:` section, keeping the same indentation. Then:

```
$ git add .github/workflows/check-files.yml
$ git commit -m "Extend CI to check services.txt"
$ git push
```

Watch it run in the Actions tab on GitHub.

Capstone Wrap-Up: Your DevOps Journey

Environment: Browser + Terminal

Have each student demonstrate:

1. Show their GitHub repo with all files
2. Show the commit history (`git log --oneline`)
3. Show a closed PR
4. Show a passing GitHub Action
5. Show branch protection on `main`

This is their portfolio piece: "I set up version control and CI for a project."

Common Beginner Mistakes

Mistake	Why It Happens	Fix
YAML indentation errors	Copied code without proper spaces	YAML uses 2 spaces (not tabs). Use an editor that shows spaces
Workflow doesn't appear	File in wrong folder	MUST be <code>.github/workflows/</code> (case sensitive)
Action fails immediately	Typo in <code>uses:</code> line	Double-check <code>actions/checkout@v4</code>
Branch protection too strict	Required checks not running yet	Make sure the workflow name matches exactly

Appendix A: Quick Reference Card

Essential Commands

Command	What It Does	When to Use
<code>git init</code>	Start Git in a folder	Beginning of a new project
<code>git status</code>	Check what's happening	ALL THE TIME. Before every commit.
<code>git add filename</code>	Put file in staging box	When a file is ready to be committed
<code>git commit -m "message"</code>	Take a snapshot	After staging
<code>git log --oneline</code>	See compact history	When you need to find an old commit
<code>git branch name</code>	Create a new branch	Starting a new feature
<code>git switch name</code>	Change branches	Moving between features
<code>git merge name</code>	Combine branches	Finishing a feature
<code>git push</code>	Upload to GitHub	Sharing your work
<code>git pull</code>	Download from GitHub	Getting the latest team code
<code>git revert COMMIT</code>	Undo a commit safely	Fixing a mistake in history
<code>git stash</code>	Save temporary work	Switching tasks urgently
<code>git stash pop</code>	Restore stashed work	Coming back to your task

The Git Workflow (Visual)



Common File Statuses in `git status`

Status	Meaning	Next Step
Untracked	Git sees it but isn't tracking	<code>git add</code>
Modified	File changed, not staged	<code>git add</code>
Staged	Ready to commit	<code>git commit</code>
Clean/Committed	Everything matches	Nothing!

Trouble? Undo!

Situation	Command
Staged wrong file	<code>git restore --staged filename</code>
Made bad edit, not staged	<code>git restore filename</code>
Bad commit, not pushed	<code>git reset --soft HEAD~1</code>
Bad commit, already pushed	<code>git revert HEAD</code>
Forgot what you did	<code>git log --oneline</code> or <code>git diff</code>

Appendix B: Instructor Timing Cheatsheet

Session	Slides	Lab	Exercise	Review	Total
1: First Repo	15 min	30 min	10 min	5 min	60 min
2: Bug Fix	15 min	30 min	10 min	5 min	60 min
3: GitHub PR	15 min	30 min	10 min	5 min	60 min
4: Revert + Conflict	15 min	30 min	10 min	5 min	60 min
5: CI/CD Actions	15 min	30 min	10 min	5 min	60 min

Ways to Shorten if Running Late

- Skip the `git stash` bonus in Session 4 (saves 5 min)

- Skip branch protection demo in Session 5 (saves 5 min)
- Do Session 3's PR exercise as a demo instead of hands-on (saves 10 min)
- Combine Session 1 and 2 if students are very technical (advanced groups only)

Appendix C: Glossary (From 5-Year-Old to Professional)

Simple Term	Professional Term	What It Means
Folder	Repository	A project that Git watches
Photo	Commit	A saved snapshot of files
Prep box	Staging Area	Files waiting to be committed
Time machine	Version Control	System that saves and restores history
Library	Remote / Origin	Server copy of the repo (GitHub)
Closed lane	Branch	A parallel line of development
Combine lanes	Merge	Joining two branches
Permission form	Pull Request	Asking to merge code into main
Robot	CI/CD Workflow	Automatic checks on code
Recipe	YAML file	Instructions for the robot

End of Approach 3: Story-Mode Curriculum

Student Handout — Git & GitHub for DevOps

All Exercises with Solutions

Works with all three approaches (Linear, Project-Driven, Story-Mode)

How to Use This Handout

1. **During class:** Try each exercise on your own first.
2. **When stuck:** Peek at the solution. Don't copy-paste without understanding.
3. **After class:** Re-do the exercises without looking. That's when learning happens.

Every exercise includes:

- The scenario
 - The commands to run
 - The expected output
 - The explanation of WHY it works
-
-

SESSION 1 EXERCISES: Your First Repository

Exercise 1.1: Create and Commit a New File

Scenario: You are starting a new project. Create a file called `tools.txt` that lists your DevOps tools.

Environment: Terminal (Git Bash / Terminal)

Task:

1. Navigate to your project folder.
2. Create `tools.txt` with the content: `Git, GitHub, CI/CD`
3. Stage it.
4. Commit it with the message: `Add tools list`
5. Check your history.

Solution:

```
$ echo "Git, GitHub, CI/CD" > tools.txt
$ git add tools.txt
$ git commit -m "Add tools list"
```

Expected Output:

```
[main 1234abc] Add tools list
1 file changed, 1 insertion(+)
create mode 100644 tools.txt
```

Verify:

```
$ git log --oneline
```

Expected Output:

```
1234abc Add tools list
def5678 Add course description
abc1234 Add welcome message
```

Why this works:

- `echo > file` writes text to a file.
- `git add` puts the file in the staging area (the box before the photo).
- `git commit -m` takes the photo and labels it.
- `git log` shows all photos in the album.

Exercise 1.2: Fix a File and Commit Again

Scenario: You realize `tools.txt` should also include `Docker` .

Task:

1. Append `Docker` to `tools.txt` .
2. Stage and commit with message: `Add Docker to tools list`
3. Check `git log --oneline` .

Solution:

```
$ echo "Docker" >> tools.txt
$ cat tools.txt
```

Expected Output:

```
Git, GitHub, CI/CD
Docker
```

```
$ git add tools.txt
$ git commit -m "Add Docker to tools list"
```

Verify:

```
$ git log --oneline
```

Why this works:

- `>>` appends (adds to the end). `>` would overwrite.
 - Every commit is a snapshot. Git remembers the old version AND the new version.
-

Exercise 1.3: Check What Changed

Scenario: You want to see what your last commit did.

Task:

1. Show the last commit's details.
2. Show what files changed in the last commit.

Solution:

```
$ git show HEAD
```

To see only files:

```
$ git show HEAD --stat
```

Why this works:

- `HEAD` means "the most recent commit."
 - `git show` displays the commit message, author, date, and the diff.
 - `--stat` shows only filenames and line counts.
-
-

SESSION 2 EXERCISES: Branching and Merging

Exercise 2.1: Create a Feature Branch

Scenario: You want to add a `contact.txt` page. Do it on a branch.

Task:

1. Create and switch to branch `feature-contact` .
2. Create `contact.txt` with: `Email: team@devops.com`
3. Stage and commit.
4. Switch back to `main` .
5. Verify `contact.txt` does NOT exist on `main` .

Solution:

```
$ git branch feature-contact
$ git switch feature-contact
$ echo "Email: team@devops.com" > contact.txt
$ git add contact.txt
$ git commit -m "Add contact page"
```

Verify main is safe:

```
$ git switch main
$ ls
```

Expected Output: (NO `contact.txt`)

```
about.txt  home.txt  tools.txt
```

Why this works:

- Branches are parallel universes. Changes on `feature-contact` do not affect `main` .
- This is the #1 reason branches exist — to protect `main` .

Exercise 2.2: Merge the Branch

Scenario: The contact page is ready. Merge it into `main`.

Task:

1. Merge `feature-contact` into `main`.
2. Verify `contact.txt` now exists.
3. Delete the branch.

Solution:

```
$ git switch main
$ git merge feature-contact
```

Expected Output:

```
Fast-forward
 contact.txt | 1 +
 1 file changed, 1 insertion(+)
```

Verify:

```
$ ls
```

Expected Output:

```
about.txt  contact.txt  home.txt  tools.txt
```

Delete branch:

```
$ git branch -d feature-contact
```

Why this works:

- `git merge` moves changes from one branch to another.
- Fast-forward means `main` was unchanged since branching — clean and simple.
- Deleting branches keeps things tidy. The commits are safely in `main`.

Exercise 2.3: Create and Merge Multiple Branches

Task:

1. Create branch `feature-skills` and add `skills.txt` .
2. Merge it.
3. Create branch `feature-services` and add `services.txt` .
4. Merge it.
5. Check `git log --oneline` .

Solution:

```
$ git branch feature-skills
$ git switch feature-skills
$ echo "Git, Docker, Kubernetes" > skills.txt
$ git add skills.txt
$ git commit -m "Add skills page"
$ git switch main
$ git merge feature-skills
$ git branch -d feature-skills

$ git branch feature-services
$ git switch feature-services
$ echo "Web Hosting, CI/CD" > services.txt
$ git add services.txt
$ git commit -m "Add services page"
$ git switch main
$ git merge feature-services
$ git branch -d feature-services
```

Verify:

```
$ git log --oneline
$ ls
```

Why this works:

- Each branch is independent. You merge them when ready.
 - This is how teams work: multiple developers, multiple branches, one `main` .
-
-

SESSION 3 EXERCISES: GitHub and Pull Requests

Exercise 3.1: Push to GitHub

Scenario: Your code is ready. Push it to GitHub.

Task:

1. Add remote `origin` pointing to your GitHub repo.
2. Push `main`.
3. Refresh GitHub and verify files are there.

Solution:

```
# Replace YOURUSERNAME with your actual username
$ git remote add origin git@github.com:YOURUSERNAME/my-repo.git
$ git push -u origin main
```

Expected Output:

```
* [new branch]      main -> main
```

Verify:

- Open `https://github.com/YOURUSERNAME/my-repo` in your browser.
- You should see your files.

Why this works:

- `origin` is a nickname for your GitHub repo. You only set it once.
- `-u` links your local `main` to remote `main`. After this, just `git push` works.

Exercise 3.2: Complete PR Workflow

Scenario: Add a `faq.txt` file using the full Pull Request workflow.

Task:

1. Create branch `feature-faq`.
2. Add `faq.txt` with a question and answer.
3. Commit and push.
4. Open a Pull Request on GitHub.
5. Review, approve, and merge it.
6. Pull locally.
7. Delete the branch.

Solution:

```
$ git branch feature-faq
$ git switch feature-faq
$ cat > faq.txt << 'EOF'
Q: What is this project?
A: My DevOps portfolio.
EOF
$ git add faq.txt
$ git commit -m "Add FAQ page"
$ git push -u origin feature-faq
```

On GitHub:

1. Click **"Compare & pull request"**
2. Title: `Add FAQ page`
3. Create PR
4. **Files changed** → Review
5. **Review changes** → Approve → Submit
6. **Merge pull request** → Confirm
7. **Delete branch**

Back in terminal:

```
$ git switch main
$ git pull
$ git branch -d feature-faq
```

Verify:

```
$ ls
```

Expected Output:

```
about.txt  contact.txt  faq.txt  home.txt  services.txt  skills.txt
```

Why this works:

- PRs are the professional way to merge code. No one pushes directly to `main`.
 - Code review catches bugs before they reach production.
-

Exercise 3.3: Pull Updates from Teammates

Scenario: A teammate pushed a change to `main` on GitHub.

Task:

1. Run `git pull` to get their changes.
2. Check `git log --oneline` to see their commit.

Solution:

```
$ git switch main  
$ git pull
```

Why this works:

- `git pull` downloads changes from GitHub AND merges them into your local `main`.
 - Always pull before starting new work. Otherwise you'll work on old code.
-
-

SESSION 4 EXERCISES: Undoing and Conflicts

Exercise 4.1: Revert a Bad Commit

Scenario: You accidentally overwrote `home.txt` with bad content.

Task:

1. Overwrite `home.txt` badly and commit it.
2. Find the bad commit ID.
3. Revert it.
4. Verify the original content is restored.

Solution:

```
$ echo "BAD CONTENT" > home.txt
$ git add home.txt
$ git commit -m "Accidentally broke homepage"
$ git log --oneline
```

Expected: Top line is the bad commit (e.g., `bad1234`).

```
$ git revert bad1234
```

An editor opens. Save and exit.

Expected Output:

```
[main revert789] Revert "Accidentally broke homepage"
```

Verify:

```
$ cat home.txt
```

Expected: Original content is back.

Why this works:

- `git revert` creates a NEW commit that undoes the bad one.
 - The bad commit stays in history (audit trail). This is the safe way.
-

Exercise 4.2: Resolve a Merge Conflict

Scenario: Two branches both edited `about.txt`. Git can't decide which to keep.

Task:

1. Change `about.txt` on `main`.
2. Create branch `update-about-1` from BEFORE that commit and change it differently.
3. Merge both into `main`.
4. Resolve the conflict.
5. Complete the merge.

Solution:

```
# Change about.txt on main
$ echo "Updated main version" > about.txt
$ git add about.txt
$ git commit -m "Update about on main"

# Create branch from before this commit
$ git branch update-about-1 HEAD~1
$ git switch update-about-1
$ echo "Updated branch version" > about.txt
$ git add about.txt
$ git commit -m "Update about on branch"

# Merge into main
$ git switch main
$ git merge update-about-1
```

Expected: `CONFLICT (content): Merge conflict in about.txt`

Resolve:

```
$ cat about.txt
# (see conflict markers)
```


Edit `about.txt` — remove `<<<<<<`, `=====`, `>>>>>>` lines, keep the content you want:

```
Main version updated with branch improvements.
```

```
$ git add about.txt
$ git commit -m "Resolve about page conflict"
```

Why this works:

- Conflicts mean Git needs human judgment. It's not broken — it's asking for help.
- After editing, you MUST stage and commit to complete the merge.

Exercise 4.3: Use Git Stash

Scenario: You're editing a file but need to switch tasks urgently.

Task:

1. Start editing a file (but don't commit).
2. Stash your work.
3. Do something else on `main`.
4. Pop your stash back.

Solution:

```
$ git switch main
$ echo "Work in progress..." >> about.txt
$ git stash
```

Expected Output:

```
Saved working directory and index state...
```

Do something else:

```
$ git status
# (clean)
```

Restore work:

```
$ git stash pop
```

Expected Output:

```
On branch main
Changes not staged for commit:
  modified:   about.txt
```

Why this works:

- Stash saves your uncommitted work temporarily.
 - The working directory becomes clean so you can switch branches or fix emergencies.
-
-

SESSION 5 EXERCISES: CI/CD with GitHub Actions

Exercise 5.1: Create a CI Workflow

Scenario: You want a robot to check that `home.txt` and `about.txt` exist on every push.

Task:

1. Create `.github/workflows/check-files.yml`.
2. Write a workflow that checks files exist.
3. Commit and push.
4. Watch it pass in the Actions tab.

Solution:

```
$ mkdir -p .github/workflows
$ cat > .github/workflows/check-files.yml << 'EOF'
name: Check Files
on:
  push:
    branches: [ main ]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Check required files
        run: |
          test -f home.txt && echo "✅ home.txt"
          test -f about.txt && echo "✅ about.txt"
          echo "All checks passed!"
EOF
$ git add .github/workflows/check-files.yml
$ git commit -m "Add CI workflow"
$ git push
```

Verify:

- Go to GitHub → **Actions** tab
- See green checkmark 

Why this works:

- YAML files in `.github/workflows/` are automatically detected by GitHub.
 - The `actions/checkout@v4` step downloads your code onto the runner.
 - `run:` executes shell commands.
-

Exercise 5.2: Watch CI Catch a Bug

Scenario: You intentionally break something and watch CI fail.

Task:

1. Create branch `break-ci`.
2. Delete `about.txt`.
3. Commit and push.
4. Open a PR and watch CI fail .
5. Close the PR without merging.

Solution:

```
$ git branch break-ci
$ git switch break-ci
$ git rm about.txt
$ git commit -m "Remove about page (should fail CI)"
$ git push -u origin break-ci
```

On GitHub:

1. Open PR
2. CI runs automatically
3. Fails with " about.txt MISSING"
4. Close PR

Why this works:

- CI catches bugs BEFORE they reach `main`.
 - Failed PRs should never be merged.
-

Exercise 5.3: Set Up Branch Protection

Scenario: Lock `main` so no one can push directly.

Task:

1. Add branch protection rule for `main`.
2. Require PR before merging.
3. Require status checks to pass.
4. Try direct push and verify you're blocked.

Solution:

On GitHub:

1. Repo → **Settings** → **Branches**
2. Add rule: `main`
3. Check:
 - Require pull request before merging
 - Require status checks to pass
 - Select `Check Files`
4. **Create**

Test in terminal:

```
$ git switch main
$ echo "test" >> home.txt
$ git add home.txt
$ git commit -m "Test direct push"
$ git push
```

Expected: Error — protected branch.

Undo test commit:

```
$ git reset --soft HEAD~1
$ git restore --staged home.txt
$ git checkout -- home.txt
```

Why this works:

- Branch protection enforces quality gates.
 - No one can accidentally break production by pushing directly.
-

BONUS EXERCISES

Bonus 1: Write a Good `.gitignore`

Scenario: You never want to commit secrets, logs, or IDE files.

Task:

1. Create `.gitignore`.
2. Add entries for `.env`, `*.log`, `.vscode/`.
3. Commit it.

Solution:

```
$ cat > .gitignore << 'EOF'
# Secrets
.env
.env.local

# Logs
*.log
logs/

# IDE
.vscode/
.idea/
EOF
$ git add .gitignore
$ git commit -m "Add .gitignore"
```

Why this works:

- `.gitignore` tells Git to never track matching files.
- **Never commit secrets.** Use `.gitignore` to protect yourself.

Bonus 2: View Blame

Scenario: A line of code is confusing. Who wrote it?

Task:

1. Run `git blame` on a file.
2. Identify who wrote each line.

Solution:

```
$ git blame home.txt
```

Expected Output:

```
abc1234 (Your Name 2024-01-15) Welcome to my portfolio!  
def5678 (Your Name 2024-01-16) I'm learning DevOps.
```

Why this works:

- `git blame` shows the commit and author for every line.
 - Essential during incidents: "Who changed this config line?"
-

Bonus 3: Tag a Release

Scenario: Your portfolio is complete. Tag it as version 1.0.0.

Task:

1. Create annotated tag `v1.0.0`.
2. Push it to GitHub.
3. View it on GitHub.

Solution:

```
$ git tag -a v1.0.0 -m "First stable release"  
$ git push origin v1.0.0
```

Verify:

- GitHub → **Releases** (right sidebar)
- See your tag

Why this works:

- Tags mark specific commits as milestones.
- Semantic versioning (v1.0.0) is the industry standard.

End of Student Handout — Good luck, engineer!

Quizzes & Assessments — Git & GitHub for DevOps

5 Session Quizzes + 1 Final Comprehensive Exam

How to Use These Assessments

- **Quiz time:** 10 minutes per session quiz, 30 minutes for final
 - **Format:** Multiple choice + practical scenarios
 - **Grading:** Immediate self-check (answers at bottom of each section)
 - **Passing:** 70% on quizzes, 80% on final
-
-

SESSION 1 QUIZ: Your First Repository

Time: 10 minutes | **Questions:** 8

Question 1

What does `git init` do?

- A) Downloads code from GitHub
 - B) Creates a new `.git` folder to start tracking a directory
 - C) Commits all files in the folder
 - D) Connects to a remote server
-

Question 2

What is the staging area?

- A) A folder where Git stores old commits
 - B) A temporary holding area for files before they are committed
 - C) The GitHub website
 - D) A backup folder
-

Question 3

Which command puts a file into the staging area?

- A) `git commit`
- B) `git push`

- C) `git add`
 - D) `git init`
-

Question 4

Which command permanently saves your staged files?

- A) `git add`
 - B) `git save`
 - C) `git commit`
 - D) `git stage`
-

Question 5

What happens if you run `git status` in a folder that is NOT a Git repository?

- A) It shows "nothing to commit"
 - B) It shows "fatal: not a git repository"
 - C) It creates a repository automatically
 - D) It lists all files in the folder
-

Question 6

Which of these is a GOOD commit message?

- A) "asdf"
 - B) "fix stuff"
 - C) "Add nginx configuration for production"
 - D) "update"
-

Question 7

What does `git log --oneline` show?

- A) All deleted files
 - B) A compact list of commits
 - C) All branches
 - D) All remote servers
-

Question 8 (Practical Scenario)

You create a file `config.txt` and want to commit it. What is the correct sequence?

Write the commands in order:

1. _____
 2. _____
 3. _____
-

Answers: Session 1

1. **B** — `git init` creates the `.git` folder and starts tracking the directory.
2. **B** — The staging area temporarily holds files before they are committed.
3. **C** — `git add` stages files.
4. **C** — `git commit` saves staged files permanently.
5. **B** — Git shows "fatal: not a git repository" if `.git` doesn't exist.
6. **C** — Descriptive messages explain what changed and why.
7. **B** — `git log --oneline` shows each commit as one line.
8. Solution:
``bash
9. `git add config.txt`
10. `git commit -m "Add configuration file"`

11. git log --oneline
```

---

---

# SESSION 2 QUIZ: Branching and Merging

---

**Time:** 10 minutes | **Questions:** 8

---

## Question 1

---

Why do we use branches?

- A) To make Git run faster
  - B) To protect `main` while we experiment safely
  - C) To delete old commits
  - D) To connect to GitHub
- 

## Question 2

---

Which command creates a new branch?

- A) `git create branch-name`
  - B) `git branch branch-name`
  - C) `git new branch-name`
  - D) `git switch branch-name`
- 

## Question 3

---

Which command switches to an existing branch?

- A) `git branch branch-name`
- B) `git change branch-name`



- C) `git switch branch-name`
  - D) `git move branch-name`
- 

## Question 4

---

What is a fast-forward merge?

- A) A merge that deletes the branch being merged
  - B) A merge where `main` hasn't changed since branching — Git just moves the pointer
  - C) A merge that runs faster than normal
  - D) A merge that doesn't require a commit message
- 

## Question 5

---

What happens if you delete a branch after merging?

- A) All its commits are lost
  - B) Its commits are safely in `main` ; the branch label is deleted
  - C) The merge is undone
  - D) Git shows an error
- 

## Question 6

---

Create and switch to a branch in one command:

- A) `git branch --create branch-name`
  - B) `git switch -c branch-name`
  - C) `git checkout --new branch-name`
  - D) `git branch -s branch-name`
-

## Question 7

---

You switch to `main` and run `ls`. A file from your feature branch is NOT there. Why?

- A) The file was deleted
  - B) Branches are isolated — changes on one branch don't appear on another until merged
  - C) Git is broken
  - D) The file is hidden
- 

## Question 8 (Practical Scenario)

---

You need to add a new `contact.txt` page via a branch. Write the complete sequence:

1. \_\_\_\_\_
  2. \_\_\_\_\_
  3. \_\_\_\_\_
  4. \_\_\_\_\_
  5. \_\_\_\_\_
  6. \_\_\_\_\_
  7. \_\_\_\_\_
- 

## Answers: Session 2

---

1. **B** — Branches let you work safely without affecting `main`.
2. **B** — `git branch branch-name` creates a branch.
3. **C** — `git switch branch-name` changes branches.
4. **B** — Fast-forward happens when `main` hasn't diverged.
5. **B** — Commits stay in history; only the branch label is deleted.
6. **B** — `git switch -c branch-name` creates and switches.
7. **B** — Branches are isolated by design.

8. Solution:

```
```bash
```

9. git branch feature-contact

10. git switch feature-contact

11. echo "Email: hello@example.com" > contact.txt

12. git add contact.txt

13. git commit -m "Add contact page"

14. git switch main

15. git merge feature-contact

16. git branch -d feature-contact

```
```
```

---

---

# SESSION 3 QUIZ: GitHub, Push, Pull, and Pull Requests

---

**Time:** 10 minutes | **Questions:** 8

---

## Question 1

---

What is a remote?

- A) A git command that undoes changes
  - B) A copy of your repository on another server (like GitHub)
  - C) A branch that is far away from `main`
  - D) A deleted commit
- 

## Question 2

---

Which command uploads your commits to GitHub?

- A) `git upload`
  - B) `git send`
  - C) `git push`
  - D) `git publish`
- 

## Question 3

---

Which command downloads the latest changes from GitHub?

- A) `git download`
- B) `git fetch`

- C) `git pull`
  - D) `git receive`
- 

## Question 4

---

What does `git remote add origin ...` do?

- A) Deletes the remote
  - B) Creates a nickname `origin` for your GitHub repo URL
  - C) Pushes code to GitHub
  - D) Creates a new branch called `origin`
- 

## Question 5

---

What is a Pull Request?

- A) A request to delete a branch
  - B) A formal request to merge your branch into main, with code review
  - C) A command to download code
  - D) An email to your manager
- 

## Question 6

---

Why shouldn't you push directly to `main` ?

- A) It's faster to use branches
  - B) Direct pushes bypass code review and can break production
  - C) GitHub doesn't allow it
  - D) Branches cost money
-

## Question 7

---

What does `git clone` do?

- A) Deletes a repository
  - B) Copies a remote repository to your local machine
  - C) Creates a new empty repository
  - D) Merges two branches
- 

## Question 8 (Practical Scenario)

---

You just merged a PR on GitHub. Your local `main` doesn't have the changes yet. Write the commands to sync:

1. \_\_\_\_\_
  2. \_\_\_\_\_
  3. \_\_\_\_\_
- 

## Answers: Session 3

---

1. **B** — A remote is a copy on a server.
2. **C** — `git push` uploads commits.
3. **C** — `git pull` downloads and merges; `git fetch` only downloads.
4. **B** — `origin` is a nickname for the remote URL.
5. **B** — PRs request merging with review.
6. **B** — Direct pushes skip quality checks.
7. **B** — `git clone` copies a remote repo locally.
8. Solution:  
``bash
9. `git switch main`
10. `git pull`

11. `git branch -d feature-branch-name`  
``

---

---

# SESSION 4 QUIZ: Undoing and Conflicts

---

**Time:** 10 minutes | **Questions:** 8

---

## Question 1

---

You need to undo a bad commit that was already pushed. What do you use?

- A) `git reset --hard`
  - B) `git revert`
  - C) `git delete`
  - D) `git erase`
- 

## Question 2

---

What does `git reset --soft HEAD~1` do?

- A) Deletes all files
  - B) Removes the last commit but keeps the changes staged
  - C) Deletes the repository
  - D) Pushes to GitHub
- 

## Question 3

---

You see this in a file:



```
<<<<<<< HEAD
my version
=====
their version
>>>>>>> their-branch
```

What is this?

- A) A bug in Git
  - B) A merge conflict that needs manual resolution
  - C) A virus
  - D) An encrypted message
- 

## Question 4

---

After fixing a merge conflict, what must you do?

- A) Delete the file
  - B) Run `git restart`
  - C) `git add .` then `git commit`
  - D) Run `git fix`
- 

## Question 5

---

What is `git stash` used for?

- A) Deleting branches
  - B) Saving uncommitted work temporarily so you can switch tasks
  - C) Pushing to GitHub
  - D) Creating a new repository
-

## Question 6

---

How do you restore stashed work?

- A) `git stash restore`
  - B) `git stash pop`
  - C) `git stash apply`
  - D) Both B and C
- 

## Question 7

---

What is `git reflog` ?

- A) A list of remote servers
  - B) A log of all Git actions (even deleted commits)
  - C) A list of branches
  - D) A log of GitHub API calls
- 

## Question 8 (Practical Scenario)

---

You committed a bad change but haven't pushed yet. Write the safe command to undo the last commit while keeping your changes:

---

## Answers: Session 4

---

1. **B** — `git revert` is safe for shared history.
2. **B** — `--soft` removes the commit but keeps changes staged.
3. **B** — These markers indicate a conflict. Remove them, edit, stage, commit.
4. **C** — Stage resolved files and commit to complete the merge.

- 5. **B** — Stash saves work-in-progress temporarily.
  - 6. **D** — `pop` applies and removes; `apply` applies but keeps it.
  - 7. **B** — Reflog records everything — your safety net.
  - 8. Solution: `git reset --soft HEAD~1`
- 
-

# SESSION 5 QUIZ: CI/CD with GitHub Actions

---

**Time:** 10 minutes | **Questions:** 8

---

## Question 1

---

What does CI stand for?

- A) Code Integration
  - B) Continuous Integration
  - C) Computer Intelligence
  - D) Central Integration
- 

## Question 2

---

GitHub Actions workflows are stored in which folder?

- A) `.github/actions/`
  - B) `.github/workflows/`
  - C) `.actions/`
  - D) `workflows/`
- 

## Question 3

---

What file format are GitHub Actions workflows written in?

- A) JSON
- B) XML

- C) YAML
  - D) HTML
- 

## Question 4

---

What does `runs-on: ubuntu-latest` mean?

- A) Your laptop must run Ubuntu
  - B) GitHub will use a virtual machine running Ubuntu
  - C) The workflow only works on Ubuntu
  - D) The branch name must be `ubuntu-latest`
- 

## Question 5

---

What does `actions/checkout@v4` do?

- A) Deletes the repository
  - B) Downloads your code onto the runner
  - C) Creates a new branch
  - D) Logs into GitHub
- 

## Question 6

---

What is branch protection?

- A) A firewall around your code
  - B) Rules that prevent direct pushes and enforce PRs + CI
  - C) A password on the repository
  - D) A backup service
-

## Question 7

---

Where do you store secrets (like API keys) in GitHub Actions?

- A) In the workflow file
  - B) In a text file in the repo
  - C) In Repo Settings → Secrets and variables
  - D) In the README
- 

## Question 8 (Practical Scenario)

---

Write a workflow that triggers on every push to `main` and prints "Deploying to production!".

```
name: _____
on:
 _____:
 branches: [_____]
jobs:
 deploy:
 runs-on: _____
 steps:
 - run: echo "Deploying to production!"
```

---

## Answers: Session 5

---

1. **B** — Continuous Integration
2. **B** — `.github/workflows/`
3. **C** — YAML
4. **B** — GitHub provides a VM (runner) running Ubuntu
5. **B** — It checks out your code onto the runner
6. **B** — Rules for `main` : PRs required, CI must pass
7. **C** — Secrets belong in Settings → Secrets, NEVER in code

#### 8. Solution:

```
yaml name: Deploy on: push: branches: [main] jobs: deploy: runs-on:
ubuntu-latest steps: - run: echo "Deploying to production!"
```

---

---

# FINAL COMPREHENSIVE EXAM

---

**Time:** 30 minutes | **Questions:** 15 | **Passing:** 80%

---

## Section A: Multiple Choice (10 questions)

---

### Question 1

Which command initializes a new Git repository?

- A) `git start`
- B) `git init`
- C) `git new`
- D) `git create`

### Question 2

What is the correct sequence?

- A) `add` → `commit` → `push`
- B) `commit` → `add` → `push`
- C) `push` → `add` → `commit`
- D) `init` → `push` → `commit`

### Question 3

You want to undo a commit that is already on GitHub. You should:

- A) `git reset --hard`
- B) `git revert`
- C) Delete the repository
- D) Email GitHub support



## Question 4

Two branches edited the same line. What does Git do?

- A) Automatically picks the first branch
- B) Creates a merge conflict and waits for a human
- C) Deletes both changes
- D) Emails the team

## Question 5

What does `git stash pop` do?

- A) Deletes all stashes
- B) Applies the latest stash and removes it
- C) Shows stash history
- D) Pushes code to GitHub

## Question 6

A Pull Request is:

- A) A request to delete a branch
- B) A formal request to merge code with review
- C) A command to download code
- D) An email to the manager

## Question 7

Where are GitHub Actions workflows stored?

- A) `.github/actions/`
- B) `.github/workflows/`
- C) `actions.yml`
- D) `ci.yml`

### Question 8

What does `git push -u origin main` do?

- A) Deletes the main branch
- B) Pushes main and sets upstream tracking
- C) Creates a new repository
- D) Pulls code from GitHub

### Question 9

Which is the safe way to undo a local commit while keeping changes?

- A) `git reset --hard HEAD~1`
- B) `git reset --soft HEAD~1`
- C) `git rm -rf .`
- D) `git undo`

### Question 10

What does branch protection prevent?

- A) Creating branches
- B) Direct pushes to main without PR or CI
- C) Opening pull requests
- D) Deleting branches

---

## Section B: Practical Scenarios (5 questions)

---

### Question 11

Write the complete sequence to set up a new repo, add a file, and push it to GitHub.

---

---

---

---

---

---

---

## Question 12

You committed a file to `main` but meant to commit to a feature branch. Write the fix.

---

---

---

## Question 13

Write a GitHub Actions workflow that runs on every pull request to `main` and prints "Running tests...".

---

---

---

---

---

---

---

---

---

## Question 14

You see a merge conflict in `config.txt`. Write the exact steps to resolve it.

1. 

---
2. 

---
3. 

---
4. 

---

### Question 15 (Bonus)

You accidentally committed a file called `secrets.env` containing a password. It has NOT been pushed. Write the complete fix.

1. \_\_\_\_\_
2. \_\_\_\_\_
3. \_\_\_\_\_
4. \_\_\_\_\_
5. \_\_\_\_\_

## Final Exam Answers

### Section A

| Q  | Answer |
|----|--------|
| 1  | B      |
| 2  | A      |
| 3  | B      |
| 4  | B      |
| 5  | B      |
| 6  | B      |
| 7  | B      |
| 8  | B      |
| 9  | B      |
| 10 | B      |

### Section B

Question 11:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin git@github.com:USER/REPO.git
git branch -M main
git push -u origin main
```

### Question 12:

```
git switch -c feature-branch # Brings the commit
git switch main
git reset --soft HEAD~1
```

### Question 13:

```
name: Test
on:
 pull_request:
 branches: [main]
jobs:
 test:
 runs-on: ubuntu-latest
 steps:
 - run: echo "Running tests..."
```

### Question 14:

1. Open config.txt, remove <<<<<< ===== >>>>>> markers
2. Edit to keep desired content
3. git add config.txt
4. git commit -m "Resolve config conflict"

### Question 15 (Bonus):

1. git reset --soft HEAD~1 # Undo last commit, keep changes staged
  2. git restore --staged secrets.env # Unstage the file
  3. echo "secrets.env" >> .gitignore # Add to .gitignore
  4. git add .gitignore
  5. git commit -m "Add .gitignore and remove secret"
- # Then change the password immediately!

## Scoring Guide

---

| Score | Grade | Meaning                 |
|-------|-------|-------------------------|
| 15/15 | A+    | DevOps ready            |
| 13-14 | A     | Strong understanding    |
| 11-12 | B     | Good, review weak areas |
| 9-10  | C     | Needs more practice     |
| < 9   | D/F   | Retake Session quizzes  |

---

*Good luck!*

---